Agentized Graph Reasoning via Policy-Guided Deduction

[1]Shenzhen Institute of Computing Sciences, China [2]University of Edinburgh, UK

ABSTRACT

This paper studies *agentized graph reasoning*, where an inference agent dynamically selects rule applications to reach a target goal without computing exhaustive logical closure. The challenge is how to choose inference steps that efficiently advance toward the goal. We propose *policy-guided deduction* and formalize the *policy selection problem* over Graph Association Rules (GARs), to minimize the cost of reaching a goal while preserving the correctness. We show that policy selection is intractable. To address this, we develop a goal-aware scoring mechanism that combines rule dependencies, graph evidence, and execution-cost estimates to prioritize inference steps. We devise deterministic and randomized score-based policies, cost-to-go learned policies, and hybrid neuro-symbolic policies, integrated into an adaptive deduction algorithm. We establish soundness, goal-level completeness under fair execution, and termination guarantees. Using real-world workloads, we empirically show that policy-guided deduction substantially reduces reasoning cost and runtime, and generalizes across diverse tasks.

PVLDB Reference Format:
. Agentized Graph Reasoning via Policy-Guided Deduction. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/SICS-Fundamental-Research-Center/agentic-graph-reasoning.

1 INTRODUCTION

Rule-based graph reasoning is widely used in data-intensive applications such as anti-money laundering (AML), fraud detection, cybersecurity, and risk analytics. It combines graph-structured data with domain knowledge to derive facts, identify risks, and generate explanations [17, 20, 21, 43, 48]. Such knowledge can be expressed as rules that infer new facts when their preconditions are satisfied. A common mechanism for rule-based deduction is the *chase* [46], which repeatedly applies applicable rules to derive new facts until no further facts can be inferred. While chase provides a well-defined logical foundation, computing all derivable facts can be prohibitively expensive when only a specific conclusion is needed.

Example 1: Consider an AML task over a financial transaction graph G, where vertices represent customers and accounts, and edges represent ownership, transfers, and shared identifiers. Suppose an analyst asks: *Why is customer v flagged as high-risk?*

The conclusion may be derived through several alternatives. One follows suspicious transfers from an account owned by v to a known high-risk account; another propagates risk through customers sharing devices or other identifiers. These alternatives may require different numbers of graph matches, intermediate deductions, and expensive machine-learning (ML) predicate evaluations.

A chase explores all these alternatives, with numerous unrelated deductions. Even when restricted to goal-relevant rules, the order of inference matters: pursuing an expensive transfer pattern before checking readily available shared-device evidence may substantially delay the conclusion. An effective reasoner should thus select inference steps that reach the target with minimal computation. □

The example highlights an important challenge: *how should a reasoning system decide which inference step to pursue next?* Different choices may establish the same goal but incur substantially different computational costs. Efficient reasoning requires not only identifying goal-relevant rules, but also selecting their applications according to the current evidence and execution costs.

Prior art and limitations. Classical chase-based systems [19, 46, 48] derive logical consequences by repeatedly applying rules, typically toward a complete closure. Goal-directed techniques, including backward chaining, magic sets [9], and tabling [13], restrict deduction to query-relevant dependencies, while cost-based planning [24, 25] optimizes execution strategies. More recently, learned graph reasoners such as NBFNet [60] and A*Net [59] use learned guidance for graph inference. However, these approaches do not directly address cost-aware selection of individual rule applications that preserve logical deduction, particularly when graph matching and ML predicate evaluation incur heterogeneous costs.

More recently, Meta-Chase [22] introduced policy-driven graph deduction, enabling adaptive selection of task-specific rule sets while establishing the semantics and consistency properties of such deduction. It provides a foundation for controlling *which rules* participate in reasoning. However, even within a selected rule set, numerous inference actions may compete to establish a goal, with different evidence requirements, costs, and potential contributions. *How to select these actions efficiently still remains open.*

Policy-guided deduction. We introduce an *inference agent* that controls deduction by selecting and evaluating rule applications toward a target goal. At each step, the agent observes the current deduction state, selects an inference action according to a policy, and updates its state based on the outcome. The policy $\pi : (S, g) \rightarrow \mu$ maps the current deduction state S and goal g to an inference action μ, *i.e.,* a grounded rule application. The agent adapts its decisions as new facts and execution evidence become available.

The agent abstraction separates logical deduction from the policy controlling its execution. Unlike fixed rule-application strategies, it allows heuristic, learned, and neuro-symbolic policies to adapt inference decisions to the current goal and accumulated execution experience. This separation enables policy optimization without

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

changing the underlying deduction semantics.

We study the *policy selection problem*: given a graph, a rule set, initial facts, and a target goal, find a policy that minimizes the *total cost of deriving the goal*, while preserving correctness. This cost includes candidate evaluation, graph matching, predicate evaluation, and policy-selection overhead. Unlike minimizing the size of a completed proof, our objective accounts for the computation on *discovering* a derivation, including unsuccessful inference attempts.

Our approach applies to rule-based deduction in general. We use Graph Association Rules (GARs) [19] as a testbed, since their combination of graph patterns, logical conditions, and ML predicates makes action selection challenging due to costly matching, heterogeneous evaluation costs, and dependencies among derived facts.

Technical challenges. The problem raises three challenges. (1) *Inference-action explosion*: a large rule set over a large graph may induce numerous candidate applications, making their enumeration and evaluation expensive. (2) *Uncertain goal-directed progress*: the usefulness of an action depends on both logical dependencies and available evidence, while its contribution to the goal may involve subsequent deductions. (3) *Efficient control with guarantees*: a policy must avoid unnecessary computation without compromising soundness or excluding necessary deductions; moreover, its decision overhead must not outweigh the savings it provides.

Our approach. We develop a *goal-aware inference scoring* mechanism that combines three complementary signals: (a) dependencies that capture an action's relevance to the goal; (b) graph evidence that estimates support for its preconditions; and (c) execution-cost estimates that capture its computational cost. These signals allow the agent to prioritize promising inferences without fully exploring their consequences. Building on these, we develop score-based policies with deterministic and randomized selection, cost-to-go learned policies, and hybrid neuro-symbolic policies. These policies guide an inference agent that adaptively selects and evaluates grounded actions. We analyze their correctness, and evaluate their effectiveness in reducing the cost of reaching target goals.

Relevance to VLDB. This work addresses graph-based inference and query optimization, both central to VLDB. Policy-guided deduction extends cost-aware query processing to rule-based graph analytics, where interdependent inference steps, costly graph matching, and ML predicate evaluation complicate execution planning. Our goal-aware policies optimize inference scheduling to reduce computational cost while preserving logical correctness.

Contributions & Organization. After reviewing graph rules and deduction in Section 2, we make the following contributions.

(1) Policy selection for graph reasoning (Section 3). We formalize the Policy Selection Problem (PSP), where an inference agent selects grounded GAR applications to derive a target goal with minimum expected execution cost. The objective accounts for policy-selection overhead, successful deductions, and failed evaluations. We establish its intractability and distinguish minimizing actual reasoning computation from optimizing a completed proof.

(2) Goal-aware scoring and adaptive deduction (Sections 4–5). We develop a scoring mechanism combining symbolic goal dependencies, reusable grounded evidence, and remaining evaluation-cost estimates. Building on it, we devise score-based, cost-to-go learned, and hybrid neuro-symbolic policies, integrated into an adaptive inference agent (AdaDeduce). We establish soundness, goal-level completeness under fair execution, and termination guarantees.

(3) Experimental study (Section 6). Using real-world and synthetic graph reasoning workloads, we empirically show that agent AdaDeduce (a) reduces end-to-end execution cost by **X–X** % and runtime by up to **X–X** × over the strongest baselines; (b) avoids **X–X** % of unnecessary inference evaluations through goal-aware action selection and evidence reuse, with only **X–X** % policy-selection overhead; and (c) maintains its advantages on unseen workloads and graphs with up to 100 million vertices and 1 billion edges, while preserving soundness and goal-level completeness.

Related work is discussed in Section 7, and Section 8 concludes the paper. Detailed proofs are provided in [1].

2 GRAPH RULES AND DEDUCTION

Thi section reviews Graph Association Rules (GARs, Section 2.1)) and chase-based deduction [19, 46] (Section 2.2). The notions define the inference actions and logical semantics underlying our policies.

2.1 Graph Association Rules

Graphs. A labeled directed graph is $G = (V, E, L, \mathcal{A})$, where V is a finite set of vertices, $E \subseteq V \times \Lambda_E \times V$ is a set of labeled edges, $L : V \to \Lambda_V$ assigns vertex labels, and $\mathcal{A}$ specifies vertex attributes. Here Λ_E and Λ_V are the edge- and vertex-label domains, respectively. The graph is schema-free: each vertex may have a different set of attributes, with no fixed schema imposed across vertices.

A *graph pattern* $Q[\bar{x}] = (V_Q, E_Q, L_Q)$ is a labeled graph whose vertices are denoted by variables $\bar{x}$. A *match* of Q in G is a homomorphism $h : V_Q \to V$ preserving labels and edges: $L_Q(x) = L(h(x))$ for all $x \in V_Q$, and $(h(x), l, h(y)) \in E$ for all $(x, l, y) \in E_Q$.

Rules. A *predicate* over $Q[\bar{x}]$ is: (a) $x.A \oplus c$ or $x.A \oplus y.B$, where $x, y \in V_Q$, A, B are attributes, c is a constant, and $\oplus \in \{=, \neq, <, >, \leq, \geq\}$; (b) $l(x, y)$, denoting an edge labeled l; or (c) $M(\bar{x})$, an ML predicate that uses a pretrained model to evaluate a condition over the attributes of $\bar{x}$, returning true or false. Unlike logical predicates, ML predicates capture data-driven relationships that are hard to specify explicitly, *e.g.,* anomalous transactions or behavioral similarity.

A *Graph Association Rule* (GAR) has the form

$$\varphi = Q[\bar{x}](X \to p_0),$$

where X is a conjunction of predicates over pattern $Q[\bar{x}]$, called the *preconditions* of φ, and p_0 is the *consequent*.

For a match h of Q in G, the *ground instance* $h(p)$ of a predicate p is obtained by replacing each variable x in p with $h(x)$. Let Γ be a set of validated ground facts. A predicate p *holds* under h *w.r.t.* Γ if: (a) for $p = x.A \oplus c$ or $x.A \oplus y.B$, the corresponding comparison on attribute values $h(x).A$ and c, or $h(x).A$ and $h(y).B$, is true and $h(p) \in \Gamma$; (b) for $p = l(x, y)$, $(h(x), l, h(y)) \in E$ or $h(p) \in \Gamma$; and (c) for $p = M(\bar{x})$, the model returns true on the attributes of $h(\bar{x})$.

The preconditions X are *satisfied* under h *w.r.t.* Γ, denoted by $\Gamma \models h(X)$, if every $p \in X$ holds. If so, the rule derives $h(p_0)$. Thus, Q identifies the graph entities, while $X \to p_0$ specifies the inference.

Example 2: Example 1 is captured by four GARs with patterns Q_1–Q_4 in Figure 1. Double circles mark variables in rule consequents;

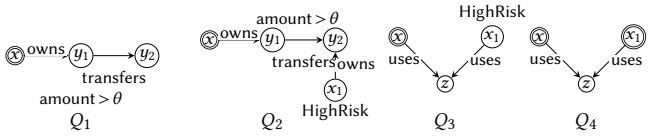


Figure 1: Patterns of the four GARs in Example 2.

ML predicates are preconditions of a rule, not part of its pattern. We write $\mathsf{HighRisk}(x)$ for the Boolean fact that customer x is high-risk.

(1) Anomalous transfers. Consider a GAR φ_1 defined with pattern $Q_1[x, y_1, y_2]$ containing edges $\mathsf{owns}(x, y_1)$ and $\mathsf{transfers}(y_1, y_2)$:

$$Q_1[x, y_1, y_2]\big(y_1.\mathsf{amount} > \theta \wedge M_{\mathsf{anom}}(y_1, y_2) \rightarrow \mathsf{HighRisk}(x)\big).$$

The rule combines three types of evidence: the graph pattern identifies a customer x owning an account y_1 that transfers funds to y_2; the logical predicate checks whether the transfer amount exceeds θ; and the ML predicate M_{anom} detects anomalous transfers. When all conditions hold, φ_1 infers that customer x is high-risk.

(2) Transfer-based risk propagation. Pattern $Q_2[x, y_1, y_2, x_1]$ adds $\mathsf{owns}(x_1, y_2)$ to the ownership and transfer edges of Q_1. GAR φ_2 is:

$$Q_2[x, y_1, y_2, x_1]\big(y_1.\mathsf{amount} > \theta \wedge \mathsf{HighRisk}(x_1) \rightarrow \mathsf{HighRisk}(x)\big).$$

Thus, if a high-risk customer x_1 receives a large transfer from an account owned by x, the rule also flags x as high-risk.

(3) Shared infrastructure. Using Q_3 with $\mathsf{uses}(x, z)$ and $\mathsf{uses}(x_2, z)$:

$$\varphi_3 : Q_3[x, x_1, z]\big(\mathsf{HighRisk}(x_1) \rightarrow \mathsf{HighRisk}(x)\big).$$

Intuitively, if customer x shares a device or facility z with a high-risk customer x_1, then the rule also flags x as high-risk, capturing risks associated with shared infrastructure.

(4) Cross-entity correlation. Using pattern $Q_4[x, x_1, z]$ that contains $\mathsf{uses}(x, z)$ and $\mathsf{uses}(x_1, z)$, GAR φ_4 is given as:

$$\varphi_4 : Q_4[x, x_1, z]\big(x \neq x_1 \wedge M_{\mathsf{sim}}(x, x_1) \rightarrow \mathsf{Collusive}(x, x_1)\big).$$

Here ML model M_{sim} checks behavioral similarity. The rule identifies possible collusion between distinct customers who share infrastructure and exhibiting similar behavior.

Together, these show that GARs combine graph structures, logical conditions, and ML predictions to derive facts. Moreover, derived facts can trigger further rules, supporting multi-step reasoning and alternative derivations across different relationships. □

2.2 Rule Applications and Deduction

Inference actions. Let Σ be a finite set of GARs and Γ_0 a set of initial validated facts over G, obtained from knowledge graphs or external sources. Write Γ for the current fact set, initially $\Gamma = \Gamma_0$.

A *candidate inference action* is $\mu = (\varphi, h)$, where $\varphi = Q[\bar{x}](X \rightarrow p_0) \in \Sigma$ and h matches its graph pattern. An action succeeds if all its preconditions $h(X)$ are satisfied *w.r.t.* Γ and $h(p_0) \notin \Gamma$; its successful application is a deduction step

$$\Gamma \Rightarrow_{\varphi,h} \Gamma \cup \{h(p_0)\}.$$

It derives a new fact $h(p_0)$ and expands Γ with $h(p_0)$.

Chase and goals. The *chase* [46] repeatedly conducts successful deduction steps from Γ_0 until no further new fact can be derived, and it produces the closure $\Gamma^* = \mathsf{Chase}(G, \Sigma, \Gamma_0)$. Since these GARs infer facts from a finite G without introducing new vertices, and inference only adds facts, the finite chase terminates and reaches the same closure regardless of application order, known as the Church-Rosser property [2, 19]. For a target fact g, *i.e.,* a *goal*, deduction succeeds once $g \in \Gamma$; computing all of Γ^* is not required.

Example 3: Consider the AML graph G in Figure 2, with customers u, u', v, accounts a_1, a_2, a_3, and shared facilities z, w. Let Γ_0 contain the ownership, transfer, and usage edges shown, the large-transfer conditions on a_1, a_3, and true ML facts $M_{\mathsf{anom}}(a_1, a_2)$ and $M_{\mathsf{sim}}(u, u')$. Rule φ_1 first derives $\mathsf{HighRisk}(u)$ using $h_1 = \{x \mapsto u, y_1 \mapsto a_1, y_2 \mapsto a_2\}$. The goal $g = \mathsf{HighRisk}(v)$ can then be derived either by φ_2, using the transfer $a_3 \rightarrow a_1$, or by φ_3, using the shared facility z. In contrast, φ_4 derives $\mathsf{Collusive}(u, u')$, which cannot contribute to g under these rules. Thus, a goal-specific deduction need not pursue every consequence computed by chase. □

The semantics above determine which facts can be derived; they do not determine which candidate action to evaluate next. To address this, Section 3 formulates that selection problem.

3 THE POLICY SELECTION PROBLEM

This section formalizes the policy-guided deduction problem. We first define the agent model (Section 3.1), and then formulate the *policy selection problem* and prove its intractability (Section 3.2). Finally, we outline our solution (Section 3.3).

3.1 Agent Model

We model deduction as an agent-controlled process that separates logical inference from policy-driven execution.

Agent-based inference control. Unlike fixed rule-application strategies, an inference agent selects actions based on the current goal, accumulated evidence, and previous execution outcomes. This separation allows heuristic, learned, and neuro-symbolic policies to control the same deduction engine without changing its logical semantics. It also enables the agent to learn from execution experience and adapt its decisions as deduction proceeds.

States and actions. Recall that an action $\mu = (\varphi, h)$ consists of a GAR φ and a match h of its graph pattern in G (Section 2.2). Let Σ be the GAR set, Γ the current validated facts, and H the execution history, recording previously evaluated actions and their outcomes. A *deduction state* is $S = (G, \Sigma, \Gamma, H)$, initially $S_0 = (G, \Sigma, \Gamma_0, \emptyset)$.

An action $\mu = (\varphi, h)$ is evaluated by checking the preconditions of φ under h. If its preconditions hold and $h(p_0) \notin \Gamma$, the action succeeds by adding $h(p_0)$ to Γ; otherwise, no new fact is derived. In either case, its outcome is recorded in H. Thus, an unsuccessful evaluation changes the execution history even when Γ is unchanged.

We use $\mathcal{A}(S)$ to denote the *available actions* at state S. It consists of all candidate actions whose consequents are not in Γ and whose previous failures do not currently rule out success.

Failure handling. The agent pays for each evaluation, and must decide whether a failed action is hopeless or merely early: discarding an early one can lose the goal, while keeping a hopeless one wastes evaluations. A precondition that reads only the graph and fixed data, *e.g.,* $a_3.\mathsf{amount} > \theta$ or $M_{\mathsf{anom}}(a_1, a_2)$ of Figure 3, stays false once false; it is *static*, and an action it blocks is discarded. One asking for a derivable fact, *e.g.,* $\mathsf{HighRisk}(u)$ in the same figure, may turn true as the fact set grows; it is *dynamic*, and the action waits for the missing facts.

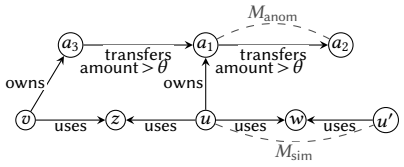


Figure 2: The graph G of Example 3; dashed curves are ML predicates.

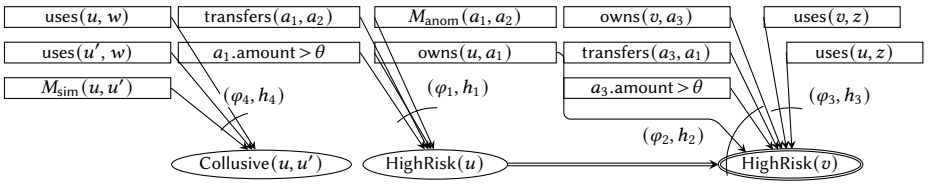


Figure 3: Proof DAG of Example 4: rectangles Γ_0 facts, ellipses derived, double ellipse the goal.

Policies and executions. Given a target fact g, a *policy* $\pi : (S, g) \to \mu$ selects an action $\mu \in \mathcal{A}(S)$. A deterministic policy selects one action, while a randomized policy specifies a distribution over $\mathcal{A}(S)$.

Starting from state S_0, a policy induces an *execution trace*

$$C = \langle (S_0, \mu_1, S_1), \ldots, (S_{k-1}, \mu_k, S_k) \rangle,$$

where state S_i results from evaluating action μ_i at S_{i-1}. The trace records both successful deductions and failed evaluations. Along a trace, a failed evaluation is treated by its kind: an action failing a static precondition leaves $\mathcal{A}(S)$ for good, while one failing only dynamic preconditions stays available and may be selected again once its missing facts are derived. An execution reaches the goal g when $g \in \Gamma_k$; we stop at its first derivation.

Example 4: Consider the goal g = HighRisk(v) in Example 3. Figure 3 shows the possible deductions. Application (φ_1, h_1) first derives HighRisk(u), which enables either (φ_2, h_2) or (φ_3, h_3) to derive g. Suppose that the agent evaluates (φ_2, h_2) before HighRisk(u) is derived. Its ownership, transfer, and amount conditions all hold, but HighRisk(u) is missing. Since this precondition is dynamic, the action waits for it, and derives g as soon as HighRisk(u) appears. Had a_3.amount > θ failed instead, the action could never derive g and would be discarded at once, since the comparison is static. In contrast, (φ_4, h_4) derives Collusive(u, u'), unrelated to g. A policy decides which applications to evaluate and in what order, determining the computation spent before reaching g. □

3.2 Policy Selection Problem

We want a policy that derives a target goal with minimum computational cost, accounting for both inference evaluation and the decisions needed to identify and select inference actions.

Execution cost. For a state S and selected action μ, let $c_{\text{sel}}(S, \mu)$ denote the cost of generating, scoring, and selecting candidates at that decision step, and $c_{\text{eval}}(S, \mu)$ the cost of evaluating action μ, including precondition checks and ML model invocations. Shared selection overhead is charged once to the step at which it occurs.

Define the cost $c(S, \mu) = c_{\text{sel}}(S, \mu) + c_{\text{eval}}(S, \mu)$.

For an execution trace $C = \langle (S_{i-1}, \mu_i, S_i) \rangle_{i=1}^{k}$, its *total cost* is

$$\text{cost}(C) = \sum_{i=1}^{k} c(S_{i-1}, \mu_i).$$

Note that both successful and failed evaluations incur cost. Thus, cost(C) measures the computation spent on *finding* a derivation, rather than the size or cost of the resulting proof.

Optimization objective. Let Π be the class of policies that select available actions, preserve sound deduction, and fairly explore candidates when necessary. For $\pi \in \Pi$, let C_π be the execution induced from state S_0 toward goal g, ending when g is first derived. A policy that fails to reach a derivable goal is assigned infinite cost.

The *Policy Selection Problem (PSP)* is: given G, Σ, Γ_0, and a derivable goal g, find a policy $\pi^* \in \Pi$ such that

$$\pi^* = \arg\min_{\pi \in \Pi} \mathbb{E}_\pi \big[\text{cost}(C_\pi)\big].$$

Here $\mathbb{E}_\pi$ denotes the expected execution cost over the possible action sequences induced by policy π. A randomized policy may select different actions at the same state, yielding executions with different costs; a deterministic policy always makes the same choice, so its expected cost equals its actual cost. In either case, decisions rely only on information available at the current state.

Example 5: Continuing Example 4, consider two policies for deriving g = HighRisk(v). One evaluates an unrelated application of φ_4 before deriving HighRisk(u) and then g. The other directly evaluates φ_1 and then φ_2. Under unit evaluation cost and zero selection overhead, the former spends three steps, while the latter spends two. More generally, expensive graph matching and ML evaluations can make poor inference choices substantially more costly, even when both policies eventually derive the same goal. □

Intractability. It is not surprising that PSP is challenging even when inference outcomes are known and all actions have unit cost.

Theorem 1: *The policy selection problem is NP-hard, even for deterministic policies over a finite set of grounded rule applications with unit evaluation costs.* □

Proof sketch: We reduce Set Cover, an NP-complete problem [23], to a restricted PSP instance. Given a collection of sets over a universe of elements, Set Cover asks whether at most k sets can be found whose union contains all elements. For each set, we use a rule deriving a fact encoding its selection; for each covered element, a rule derives its coverage fact from a selected set. Other rules combine all coverage facts into the target goal. Each selection and coverage step incurs unit cost, and the number of combination steps is fixed. Thus, a goal-reaching execution of cost at most B exists iff the Set Cover instance admits a cover within the corresponding bound. These rules can be written as GARs. The full proof is in [1]. □

The result shows that minimizing goal-reaching cost is computationally difficult even without uncertain evaluations, expensive ML predicates, or policy overhead. It suggests that we develop efficient heuristic selection rather than exhaustive optimization.

3.3 Solution Overview

To address PSP, we develop a policy-guided deduction framework that estimates which inference actions are most promising and executes them adaptively, without enumerating all possible derivations. The framework consists of the following three steps.

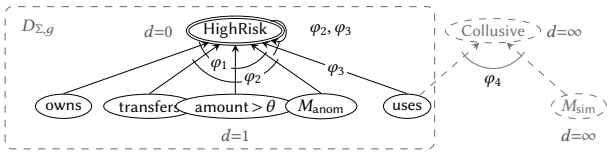


Figure 4: The dependency graph $D_{\Sigma,g}$ of Example 2; an arc marks a rule (AND), and $d(\cdot)$ the backward distance from HighRisk

(1) Goal-relevant candidate generation. We analyze the rule dependencies backward from the target goal g to identify predicates and rules that may contribute to its derivation. This analysis excludes rules that cannot lead to g, and generates candidate actions only for the remaining rules.

(2) Goal-aware inference scoring. For each candidate μ, we estimate its usefulness from three complementary signals: (a) *symbolic relevance*, for its possible contribution to g via rule dependencies; (b) *graph evidence*, estimating support for its instantiated precondition; and (c) *execution cost*, estimating the computation required to evaluate it. We use a scoring function $f(S, g, \mu)$ to combine these signals to rank candidates without fully evaluating their preconditions or downstream derivations. Section 4 develops this mechanism.

(3) Policy-guided execution. The agent selects candidates using the scores, evaluates their premises, and updates the deduction state with new facts and evaluation outcomes. We develop deterministic and randomized policies, as well as learned and hybrid neuro-symbolic policies that exploit previous execution experience. Their execution strategies preserve logical soundness and goal reachability under fair exploration, while reducing unnecessary evaluations. Section 5 presents the algorithms and their guarantees.

Example 6: Consider the goal $g = \text{HighRisk}(v)$ in Example 3. Symbolic analysis identifies GARs φ_1–φ_3 as relevant to g, excluding φ_4, which derives only Collusive facts. Initially, neither φ_2 nor φ_3 can derive g without $\text{HighRisk}(u)$. The agent thus prioritizes an application of φ_1 that derives $\text{HighRisk}(u)$. With this fact available, both φ_2 and φ_3 can derive g. Evidence-aware scoring favors φ_2 when its transfer conditions are already validated, and can avoid additional checks for φ_3. The agent then applies GAR φ_2 and reaches the goal g, without exploring unrelated deductions. □

4 GOAL-AWARE INFERENCE SCORING

This section develops a scoring function $f(S, g, \mu)$ to guide the agent in selecting inference actions toward goal g. Ideally, the agent should favor actions that are likely to advance the deduction at low cost. However, determining whether an action succeeds may itself require expensive graph matching and ML evaluation. We thus estimate its promise using three signals: *goal relevance* from rule dependencies, *grounded evidence* accumulated during execution, and *estimated evaluation cost*. Together, these signals provide an effective guidance for the policies to be developed in Section 5.

Goal relevance. Not every valid deduction helps establish the goal. We first identify potentially useful actions from the dependencies among GARs. Let $\text{pred}(p)$ denote the predicate symbol of a predicate or ground fact p. We construct an auxiliary structure for guiding reasoning, a *dependency graph* $D_{\Sigma,g}$ exemplified in Figure 4, with predicate symbols as nodes and an edge $q \to p$ labeled by φ whenever a GAR $\varphi \in \Sigma$ uses predicate q in its preconditions to derive predicate p. Pattern edges are treated as extensional preconditions. Backward reachability from $\text{pred}(g)$ identifies the predicates and rules that may contribute to g.

Let $d(q)$ be the shortest-path distance from predicate q to $\text{pred}(g)$ in $D_{\Sigma,g}$, and $d(q) = \infty$ if no such path exists. For an action $\mu = (\varphi, h)$ with consequent p_0, define its *symbolic relevance* as

$$r(g, \mu) = \begin{cases} \gamma^{d(\text{pred}(p_0))}, & d(\text{pred}(p_0)) < \infty, \\ 0, & \text{otherwise}, \end{cases}$$

where $\gamma \in (0, 1)$ discounts indirect dependencies. An action whose consequent has no dependency path to the goal cannot help derive it. Conversely, finite distance indicates only potential relevance: the particular grounding may still be unrelated to the target entity.

The result below gives us a safe, inexpensive filter to prune candidate actions before evaluating grounded preconditions.

Proposition 2: *Excluding actions with $r(g, \mu) = 0$ preserves the derivability of g.* □

Proof sketch: If an action contributes to deriving g, its consequent supports a sequence of rule applications ending at $\text{pred}(g)$. This sequence induces a path in $D_{\Sigma,g}$, so its relevance cannot be zero. □

Example 7: Consider the goal $g = \text{HighRisk}(v)$ in Example 3. Figure 4 depicts the dependency graph of the four GARs given Example 2. It shows that φ_1–φ_3 can contribute to HighRisk, while φ_4 derives only Collusive facts. Thus, applications of φ_4 are excluded. However, both $\mu_2 = (\varphi_2, h_2)$ and $\mu_3 = (\varphi_3, h_3)$ can derive the goal g. Their grounded evidence provides the missing signal. □

Evidence-based estimation. Even among goal-relevant actions, some may have most of their preconditions already established, while others still require substantial new evaluation. An unsuccessful action can also reveal useful information: its graph pattern and several preconditions may have been validated before another precondition failed. Thus, we want to exploit such observations to avoid repeatedly checking the same evidence.

For action μ, let P_μ be the set of its distinct grounded preconditions, including matched pattern edges. Let $W(S)$ contain the validated facts in Γ and positive precondition outcomes recorded in execution history H. The *evidence support* of μ is

$$e(S, \mu) = \frac{|P_\mu \cap W(S)|}{|P_\mu|}.$$

This measures the fraction of its preconditions already witnessed. Computing such overlaps for many candidates requires repeated fact lookups and can be expensive. Thus we use hyperdimensional computing (HDC) [31] to obtain a compact estimate.

HDC represents symbols by high-dimensional vectors and combines them into representations of structured objects. We assign each predicate or entity symbol a a random bipolar vector $\mathbf{v}_a \in \{-1, +1\}^D$. For a ground fact $p(a_1, \ldots, a_k)$, define

$$\mathbf{v}_{p(\bar{a})} = \text{norm}\left(\mathbf{v}_p + \sum_{i=1}^{k} \rho^i(\mathbf{v}_{a_i})\right),$$

where ρ is a fixed coordinate permutation, and norm normalizes the vector to unit length. The permutations preserve argument positions, distinguishing, *e.g.*, $\text{transfers}(a_1, a_2)$ from $\text{transfers}(a_2, a_1)$.

We bundle the observed facts into an *evidence vector* $\mathbf{E}(S)$ and the grounded preconditions of μ into a *candidate vector* $\mathbf{v}_\mu$:

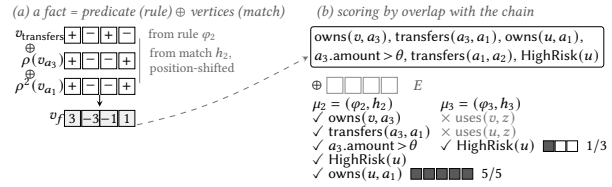


Figure 5: HDC scoring of Example 8: (a) facts bundle predicates and shifted vertices; (b) candidates score by cosine with E

$$\mathbf{E}(S) = \sum_{p \in W(S)} \mathbf{v}_p, \quad \mathbf{v}_\mu = \sum_{p \in P_\mu} \mathbf{v}_p.$$

As deduction proceeds, newly validated facts are added to $\mathbf{E}(S)$ incrementally, without rebuilding the representation. Because similar vectors have large inner products, the overlap between the two bundles provides an estimate of witnessed preconditions:

$$\widehat{e}(S, \mu) = \mathrm{clip}_{[0,1]}\left(\frac{\langle \mathbf{v}_\mu, \mathbf{E}(S)\rangle}{|P_\mu|}\right),$$

where $\mathrm{clip}_{[0,1]}$ restricts the estimate to $[0, 1]$. Thus, one vector operation summarizes the evidence supporting a candidate, without evaluating its remaining preconditions.

The result below shows when the compact representation makes accurate evidence estimates: interference between distinct fact vectors are small. Note that similarity is used only for ranking; logical deductions still require exact validation of their preconditions.

Proposition 3: *Suppose distinct encoded facts satisfy* $|\langle \mathbf{v}_p, \mathbf{v}_q\rangle| \le \epsilon$. *Then* $|\widehat{e}(S, \mu) - e(S, \mu)| \le |W(S)|\epsilon$. □

Proof sketch: Each fact shared by P_μ and $W(S)$ contributes one to the inner product. Each distinct pair contributes at most ϵ in absolute value. Summing these contributions and dividing by $|P_\mu|$ gives the bound; clipping cannot increase the error. □

Example 8: Continue Example 7. Suppose the agent first probes action μ_2 before HighRisk(u) is known. The action fails to derive the goal g, but its evaluation validates the ownership, transfer, and amount conditions. After $\mu_1 = (\varphi_1, h_1)$ derives HighRisk(u), all preconditions of μ_2 have been witnessed. In contrast, fewer preconditions of μ_3 have been examined. As illustrated in Figure 5, the accumulated evidence now favors μ_2. Thus, even an unsuccessful evaluation can improve subsequent inference decisions. □

Evaluation-cost estimation. Most preconditions of an action may already be validated, yet the few that remain can still be costly to check, *e.g.*, an ML predicate or a graph match. We estimate the remaining work rather than charging for preconditions whose outcomes are already known. Let $U(S, \mu) \subseteq P_\mu$ contain the preconditions whose outcomes have not been established from Γ or H. For each grounded precondition p, let $\widehat{T}(p)$ be its estimated evaluation cost, obtained from previous executions or a static estimate. Define

$$\widehat{c}_{\mathrm{eval}}(S, \mu) = \sum_{p \in U(S,\mu)} \widehat{T}(p).$$

The *evaluation-cost estimate* reflects heterogeneous evaluation costs and the benefit of reusing validated results. Matching costs that are already incurred are not counted again, while candidate generation and scoring overhead remain part of c_{sel} in Section 3.2.

Combined scoring. The three signals complement one another: symbolic relevance identifies potentially useful actions, evidence support estimates how much is already known, and evaluation cost estimates the remaining computation. We combine them as

$$f(S, g, \mu) = \alpha\, r(g, \mu) + \beta\, \widehat{e}(S, \mu) - \lambda\, \widehat{c}_{\mathrm{eval}}(S, \mu),$$

where $\alpha, \beta, \lambda \ge 0$ control their relative weights. Actions with $r(g, \mu) = 0$ are excluded; the remaining actions are ranked by f.

In Example 8, both μ_2 and μ_3 can derive g. However, the previous probe of μ_2 supplies reusable evidence, reducing its remaining evaluation cost. The combined score thus favors μ_2 when these advantages outweigh any differences in symbolic relevance.

Complexity. Building $D_{\Sigma,g}$ and computing $d(\cdot)$ take time linear in the dependency graph size. For dimension D, each evidence update takes $O(D)$ time, and a candidate with a precomputed vector is scored in $O(D + |P_\mu|)$ time, including cost estimation. Candidate generation and vector construction are charged separately under PSP. Thus, the score avoids repeated full evaluation of candidate preconditions by adding bounded per-candidate overhead.

5 POLICY CONSTRUCTION AND EXECUTION

This section develops goal-aware policies for the inference agent and an algorithm for their adaptive execution. Section 5.1 constructs score-based, learned, and hybrid policies using the scores of Section 4. Section 5.2 presents the deduction algorithm and shows its soundness, goal-level completeness, and termination guarantees.

5.1 Policy Construction

A scoring function ranks individual inference actions, while a policy determines which action the agent evaluates next. We first construct policies directly from $f(S, g, \mu)$, and then develop learned policies that exploit previous executions to estimate downstream costs. All policies operate on the available actions $\mathcal{A}(S)$ of Section 3.1.

Score-based policies. A *deterministic policy* selects the action that has the highest score, *i.e.,* we select action μ such that

$$\pi_{\mathrm{det}}(S, g) = \arg\max_{\mu \in \mathcal{A}_g(S)} f(S, g, \mu),$$

where $\mathcal{A}_g(S) = \{\mu \in \mathcal{A}(S) \mid r(g, \mu) > 0\}$ contains the goal-relevant available actions. Ties are resolved by a fixed order. The policy favors actions with stronger symbolic relevance and supporting evidence, but lower estimated evaluation costs.

Since f is an estimate, the score may favor an action that turns out to be costly or unsuccessful; thus the agent should also explore other promising alternatives. A *randomized policy* does so by selecting actions with probabilities proportional to their scores:

$$\pi_{\mathrm{rnd}}(\mu \mid S, g) = \frac{\exp(f(S, g, \mu)/\tau)}{\sum_{\mu' \in \mathcal{A}_g(S)} \exp(f(S, g, \mu')/\tau)},$$

where $\exp(x) = e^x$ is the exponential function, assigning higher selection probabilities to higher-scoring actions, and $\tau > 0$ controls this preference. Small τ favors the highest-scoring action, and larger τ promotes exploration. Randomization reduces reliance on potentially inaccurate scores, but does not alone guarantee completeness.

Example 9: Consider the AML goal g = HighRisk(v) of Example 3. After action $\mu_1 = (\varphi_1, h_1)$ derives HighRisk(u), both $\mu_2 = (\varphi_2, h_2)$ and $\mu_3 = (\varphi_3, h_3)$ can derive the goal g. If previously validated transfer conditions give μ_2 a higher score, then the deterministic policy selects μ_2. In contrast, the randomized policy favors μ_2 but

may explore μ_3, which uses shared infrastructure instead. □

Learned policies. The handcrafted score considers immediate relevance, evidence, and evaluation cost, but cannot fully capture the downstream computational consequences of selecting an action. We thus learn from execution experience. Let $\Phi(S, g, \mu)$ denote the features of a candidate action, including its symbolic relevance $r(g, \mu)$, evidence estimate $\widehat{e}(S, \mu)$, remaining evaluation-cost estimate $\widehat{c}_{\text{eval}}(S, \mu)$, and the representations $\mathbf{v}_\mu$ and $\mathbf{E}(S)$ from Section 4.

Rather than predicting whether an action appears in a successful derivation, we learn its *action-conditioned cost-to-go*. Let π_b be a continuation policy used to collect training executions. Define

$$Q^{\pi_b}(S, g, \mu) = \mathbb{E}\left[\text{cost}(C) \mid S, \mu, \pi_b\right],$$

where C starts by evaluating μ at state S and then follows policy π_b until reaching g. Here $\mathbb{E}$ averages the execution costs over the possible action sequences induced by π_b, since randomized choices may lead to different costs. For a deterministic π_b, the expectation reduces to the cost of its single execution. The cost includes all action evaluations and policy overhead, as in PSP. An execution that fails to reach a derivable goal has infinite cost.

To implement this policy, we train a model $\widehat{Q}_\theta$ to estimate this cost from $\Phi(S, g, \mu)$. For training, we collect completed executions under π_b, retaining the state, selected action, and remaining observed execution cost at each decision. Additional rollouts from recorded states evaluate alternative actions and their subsequent executions. These observations provide action-conditioned cost targets; unobserved alternatives are not assigned artificial costs.

For each training observation (S, g, μ, y), let y be the observed remaining execution cost after selecting μ at state S and following π_b. We train a model $\widehat{Q}_\theta(S, g, \mu)$, parameterized by θ, to predict this cost from the features $\Phi(S, g, \mu)$. The model minimizes

$$\mathcal{L}(\theta) = \sum_{(S,g,\mu,y)\in\mathcal{D}} \left(\widehat{Q}_\theta(S, g, \mu) - y\right)^2,$$

where $\mathcal{D}$ contains the collected finite-cost training observations. Note that failed goal-reaching rollouts are recorded separately rather than assigned finite targets. During execution, the agent combines learned selection with *fair* exploration: it periodically selects the longest-waiting available action, ensuring that no candidate is indefinitely ignored because of a low predicted score. Together with the revival of dynamically suspended actions, this preserves goal-level completeness (see Theorem 5 shortly).

Unlike immediate scoring, cost-to-go learning can recognize that a seemingly inexpensive action may trigger costly subsequent deductions. However, $\widehat{Q}_\theta$ estimates costs under the continuation policy π_b; it does not guarantee globally optimal policy selection.

Hybrid neuro-symbolic policies. Learned policies may assign high scores to actions that are unlikely to contribute to the target goal. To prevent such choices, we combine symbolic reasoning with learned action selection in two stages.

First, we use the rule dependency graph to restrict the available actions to $\mathcal{A}_g(S)$. By Proposition 2, excluding actions with $r(g, \mu) = 0$ preserves goal derivability. Second, among the remaining candidates, the agent selects the action with the lowest predicted cost-to-go $\widehat{Q}_\theta(S, g, \mu)$. Alternatively, it can rank candidates using the handcrafted score $f(S, g, \mu)$. Thus, symbolic filtering identifies potentially useful deductions, while learned guidance estimates which action can reach the goal at lower computational cost.

We also support LLM-generated selection guides, *e.g.*, with a model like Jev [53], for goals involving new rules or domains with limited execution experience. Once per goal, the LLM reads the GARs, available graph statistics, and the goal g, and uses its domain knowledge to suggest initial preferences among goal-relevant rules. These preferences exploit rule semantics not explicitly captured by dependency distances or HDC evidence overlap, and serve as soft initial preferences for star expansion. The online policy continues to select actions within each star using f or $\widehat{Q}_\theta$. Their influence is reduced as execution observations accumulate. The guide supplements the existing relevance, evidence, and cost signals; its benefit in unfamiliar workloads requires empirical evaluation. It is generated before deduction, without an LLM call at each action-selection step. Details are in [1].

Learned predictions and LLM-generated preferences affect only execution order. The guide removes no candidate, and every derived fact still requires exact GAR validation. Together with fair execution and revival of suspended actions, this preserves soundness and goal-level completeness.

5.2 Policy-Guided Deduction

We integrate the policies into an adaptive deduction algorithm. AdaDeduce derives the goal in two loops: a *main loop* that drives the deduction toward the goal under the policy, and a *fixpoint loop* that completes the inference steps that had to wait for premises still to be derived. It stops when the goal is derived, or when no further fact can be derived.

Working sets. Recall from Section 3.1 the deduction state $S = (G, \Sigma, \Gamma, H)$. The agent maintains two collections on top of it. The *ready set* $\mathcal{R}$ consists of pairs (φ, h), where h is a complete match of the pattern of a GAR φ; these are the discovered goal-relevant actions awaiting evaluation. The *suspended set* $\mathcal{W}$ consists of such pairs annotated with the dynamic preconditions still missing, together with the preconditions already validated; these are the partial results that wait for their missing facts. The two sets exchange objects in a closed loop: graph matching instantiates goal-relevant rules into actions, which enter $\mathcal{R}$; predicate evaluation then checks the preconditions of an action of $\mathcal{R}$ under its match, deriving its consequent into Γ, or suspending it in $\mathcal{W}$ when only dynamic preconditions are missing; and new facts complete suspended actions.

Agent. We implement AdaDeduce in Algorithm 6. Given a graph G, GARs Σ, initial facts Γ_0, and target goal g, the agent builds $D_{\Sigma,g}$ once (line 1), marking each node as a *leaf* if it has no incoming edge, and *derivable* otherwise; the two loops below run on $D_{\Sigma,g}$ and its marks. It derives g in the main loop and the fixpoint loop, and returns whether g is derived, the resulting facts, the complete execution trace, and the actual execution cost.

The main loop. The agent works in rounds over $D_{\Sigma,g}$, one star per round, each round in two stages: the generation stage produces the candidate actions of the star, and the policy stage evaluates them before the next round begins (lines 2–11).

(1) The generation stage. This stage produces all candidate actions of the rules in $D_{\Sigma,g}$. It expands one *star* of $D_{\Sigma,g}$ at a time, a rule with its consequent as the center and its preconditions as the spokes

Input: Graph G, GARs Σ, initial facts Γ_0, goal g, policy π.
Output: Trace C, facts Γ, cost c.

1. $\Gamma \leftarrow \Gamma_0$; $H, \mathcal{R}, \mathcal{W} \leftarrow \emptyset$; $c \leftarrow 0$; build $D_{\Sigma,g}$;
2. **for each** star φ of $D_{\Sigma,g}$ **do**
3. match the pattern of φ in G, reusing matches of shared sub-patterns, and add the actions (φ, h) to $\mathcal{R}$;
4. **while** $\mathcal{R} \neq \emptyset$ **do**
5. select $\mu \in \mathcal{R}$ by π;
6. check the preconditions of μ, reusing outcomes in H;
7. **if** all preconditions hold **then** $\Gamma \leftarrow \Gamma \cup \{h(p_0)\}$, where $\mu = (\varphi, h)$;
8. charge c_{sel} and c_{eval} to c; **if** $h(p_0) = g$ **then return** (C, Γ, c);
9. **else if** a leaf precondition p fails **then** drop μ, record p in H, and prune all actions sharing p;
10. **else** suspend μ in $\mathcal{W}$ with its validated and missing preconditions;
11. $S' \leftarrow (G, \Sigma, \Gamma, H)$; $C \leftarrow C \circ \langle (S, \mu, S') \rangle$;
12. **repeat**
13. **for each** $\mu \in \mathcal{W}$ **do**
14. **if** the missing preconditions of μ are all in Γ **then** apply μ as in lines 7–8 and 11, and remove it from $\mathcal{W}$;
15. **else if** some missing precondition of μ has no remaining action deriving it **then** drop μ;
16. **until** $\mathcal{W} = \emptyset$ or a round derives no new fact;
17. **return** (C, Γ, c).

Figure 6: Agent AdaDeduce: policy-guided deduction with star expansion and fixpoint revival.

(Figure 4), by matching its pattern in G; each complete match h forms one candidate action (φ, h) and enters $\mathcal{R}$ (lines 2–3). Matching checks vertex labels and edges only, and shared sub-patterns are matched once and reused.

(2) The policy stage. As the downstream stage of each generation stage, it turns the candidate actions of $\mathcal{R}$ into facts under the control of the policy π. Whenever $\mathcal{R}$ is nonempty, the agent evaluates its actions until $\mathcal{R}$ turns empty (lines 4–11): the policy picks the next action (line 5), and the agent checks its preconditions, *i.e.,* predicates, reusing outcomes in H (line 6). The marks of $D_{\Sigma,g}$ split the actions into two kinds (Section 3.1): a *static* action has only leaf spokes and is settled by one evaluation; a *dynamic* action has a derivable spoke and may have to wait. Three outcomes are possible. (a) All preconditions hold: the action derives its consequent, Γ is updated, and the cost is charged, stopping at once if the fact is g (lines 7–8). (b) A leaf precondition fails: the action can never succeed and is discarded, with every action sharing that failed precondition (line 9). (c) Only derivable facts are missing: the action is suspended in $\mathcal{W}$ with its bindings and validated preconditions (line 10). In each case the transition is recorded in the trace (line 11), and the agent proceeds to the next action of $\mathcal{R}$.

The fixpoint loop. When the main loop ends, $\mathcal{W}$ may still hold partial results waiting for missing facts, and the fixpoint loop closes them out in rounds (lines 12–13). No matching is needed anymore: all matches were computed in the main loop, and G never changes; each round only looks up the missing facts of $\mathcal{W}$ in Γ, reusing the recorded partial results. An action whose missing facts have all arrived succeeds and leaves $\mathcal{W}$ (line 14), possibly completing others in later rounds; one waiting on a fact that no remaining action can derive is abandoned (line 15). The rounds iterate until $\mathcal{W}$ is empty or Γ stays the same for two consecutive rounds, *i.e.,* it reaches a fixpoint (line 16): only a new fact could complete a partial result, so nothing in $\mathcal{W}$ can ever succeed again. Together with the early stop at g (line 8), AdaDeduce terminates, returning the trace, facts and cost (line 17).

Example 10: Continue Example 9, with $g = \mathsf{HighRisk}(v)$. Suppose that the generation stage expands the star of φ_2 first, and the policy stage picks $\mu_2 = (\varphi_2, h_2)$ from $\mathcal{R}$. Its ownership, transfer, and amount conditions hold, but $\mathsf{HighRisk}(u)$ is missing. Since only a dynamic precondition fails, the action is suspended in $\mathcal{W}$; its validated preconditions are recorded with it as a partial result. When the star of φ_1 is expanded, the agent evaluates $\mu_1 = (\varphi_1, h_1)$, deriving $\mathsf{HighRisk}(u)$. After all stars are processed, the fixpoint loop revisits μ_2 in its first round: the missing precondition is now in Γ, the recorded partial result is reused without re-evaluation, and μ_2 derives $g = \mathsf{HighRisk}(v)$. Execution stops immediately.

A learned policy may revisit μ_2 before μ_3 in the fixpoint loop because its estimated remaining cost is lower. The earlier unsuccessful evaluation is thus useful: its evidence reduces the cost of reaching the goal. □

Correctness guarantees. We next show that policy-guided deduction preserves logical validity and derives every reachable goal under fair execution. These guarantees hold due to exact GAR evaluation and the working-set discipline, not on scoring accuracy.

We first show the soundness: heuristic, learned, or LLM-based selection cannot introduce unsupported deductions.

Theorem 4: *For any policy π, every fact derived by* AdaDeduce *belongs to* $\Gamma^* = \mathsf{Chase}(G, \Sigma, \Gamma_0)$. □

Proof sketch: We show by induction on the execution steps that $\Gamma \subseteq \Gamma^*$. Initially, $\Gamma = \Gamma_0 \subseteq \Gamma^*$. For the inductive step, note that each successful action validates all preconditions of a GAR and performs a sound chase step. Failed actions add no facts. □

We next show the goal-level completeness, *i.e.,* goal reachability even when the policy's scores are inaccurate.

Theorem 5: *For finite G and Σ, under complete goal-relevant candidate discovery and fair execution,* AdaDeduce *derives all $g \in \Gamma^*$.* □

Proof sketch: We prove this also by induction on the number of chase derivation steps. Since $g \in \Gamma^*$, a finite chase derivation of g exists. By Proposition 2, every action needed for this derivation survives symbolic filtering. Every action of the derivation is generated and evaluated, since the generation stage expands every star in the cone of g. Such an action either succeeds or suspends only on dynamic preconditions; those are derived by earlier actions of the derivation, which succeed by induction, and the fixpoint rounds revisit the action until they arrive. The fixpoint cannot end before g is derived: the first action of the derivation not yet succeeded would have all its preconditions in Γ and would derive a new fact. □

Finally, we show that our algorithm terminates, and repeated unsuccessful evaluations cannot cause nontermination.

Theorem 6: *For finite G and Σ,* AdaDeduce *terminates.* □

Proof sketch: The finite graph admits finitely many grounded rule applications and ground facts. Each star is expanded once, so each action is generated once, and the policy stage evaluates each

Dataset	Domain	#vertices	#edges	#relations	avg. deg.
FB15k-237	Encyclopedic	14.5K	272K	237	18.7
ogbl-biokg	Biomedical	93.8K	4.76M	51	50.8
FinReflectKG	Financial	1.0M	17.5M	3.0K	17.5
DBpedia	Encyclopedic	5.0M	20M+	600	4.0
Synthetic	Synthetic	100M	1B	20	10.0

Table 1: Datasets ($K = 10^3$, $M = 10^6$, $B = 10^9$).

generated action once. In the fixpoint loop, every round except the last derives at least one new fact; since Γ grows monotonically and only finitely many facts exist, the number of rounds is finite. □

Computational cost. The execution cost of a trace is accounted as in Section 3.2. By Section 4, scoring a candidate with a precomputed representation takes $O(D + |P_\mu|)$ time, where D is the HDC dimension. Generation matches only the rules of $D_{\Sigma,g}$, each star exactly once, and the fixpoint loop re-checks only the missing preconditions of suspended actions, at most $|\curlywedge\mathcal{W}|$ lookups per round. Incremental maintenance of the working sets avoids repeating unaffected graph matches, and recorded precondition outcomes avoid redundant evaluations. The saving depends on the reduction in inference work relative to these control costs, which we measure experimentally.

Remark. Unlike Meta-Chase [22], which controls rule-set selection, we select individual grounded inference actions to minimize goal-reaching cost. Its novelty lies in combining cost-to-go learning, reusable execution evidence, and cone-restricted star expansion over the working sets with sound, complete, and terminating GAR deduction.

6 EXPERIMENTAL STUDY

Using real-world and synthetic graphs, this section evaluates AdaDeduce and answers six questions: (1) Does it reduce the cost of reaching inference goals? (2) How effective and accurate is goal-aware scoring? (3) How do different inference policies compare? (4) Does adaptive execution preserve correctness while avoiding redundant computation? (5) How well does it generalize and scale? (6) Which components contribute to its effectiveness?

Experimental setup. We start with the datasets, deduction workloads, baselines, evaluation metrics, and implementation.

Datasets. We use four real-world knowledge graphs and synthetic graphs, summarized in Table 1: (1) FB15k-237 [52] contains encyclopedic facts and supports controlled policy and scoring comparisons; (2) ogbl-biokg [30] is a biomedical graph with diverse entity types and relations. (3) FinReflectKG [3] contains financial knowledge extracted from corporate filings and supports risk-oriented reasoning. (4) DBpedia [35] provides a larger encyclopedic graph for scalability evaluation. We additionally generate synthetic graphs using gMark [5], with up to 100M vertices and 1B edges, to vary graph size and rule dependencies systematically.

Rule libraries and workloads. For each real-world dataset, we construct a library Σ of GARs using the discovery algorithm of [18], with minimum support **X** and confidence **X**. We retain **X** rules per dataset, and include **X** ML predicates with measured evaluation costs. These yield libraries with **X**, **X**, **X** and **X**GARs, respectively.

A deduction task is (G, Σ, Γ_0, g), as defined in Section 3. We construct derivable tasks by withholding target facts from the initial facts Γ_0 and verifying their derivability using unrestricted chase. We also construct non-derivable tasks for correctness testing. For each dataset, we evaluate **X** derivable and **X** non-derivable goals. We vary deduction depth, the number of alternative derivations, candidate-action count, and the proportion of expensive preconditions to investigate different reasoning conditions.

Baselines and policy variants. We implement agent AdaDeduce in **X**. We compare five methods: (1) *Chase*, which uses all GARs in a fixed order and computes the complete closure, serving as a full-computation reference for the upper end of inference work; (2) *DepOnly*, which uses only the dependency graph $D_{\Sigma,g}$ to guide deduction by restricting it to goal-relevant rules, evaluates their actions in a fixed order without evidence- or cost-aware ranking, and stops when the goal is derived; (3) *Deterministic*, an AdaDeduce variant that selects the ready action with the highest combined score $f(S, g, \mu)$; (4) *Random*, an AdaDeduce variant that samples ready actions from the score-based softmax distribution π_{rnd}; and (5) *Learned*, an AdaDeduce variant that selects the ready action with the lowest predicted cost-to-go $\widehat{Q}_\theta(S, g, \mu)$. Deterministic, Random, and Learned share goal-relevance filtering, candidate generation, precondition reuse, suspended-action handling, and the same fair-execution mechanism; they differ only in action selection. All five methods use the same rule semantics and exact precondition evaluation. Unless stated otherwise, AdaDeduce denotes the Learned variant in the experiments.

We additionally compare with systems that, like AdaDeduce, accelerate rule-based deduction on graphs: MiniClean [6, 7] evaluates graph rules in parallel on GPUs, and HyperBlocker [58] speeds up rule evaluation for entity blocking. Both follow exact rule semantics, so we test them on the same deduction tasks. We also compare with learned graph-reasoning systems, including RNNLogic [44], AnyBURL [40], and MINERVA [14], on tasks supported by their respective semantics. These systems optimize prediction accuracy rather than goal-reaching cost; their results are reported separately when their objectives differ from PSP.

Metrics. Our primary metric is *total execution cost*, including candidate generation, policy selection, graph matching, precondition evaluation, ML invocations, and candidate maintenance, as defined in Section 3. In addition, we measure end-to-end runtime, the numbers of evaluated and failed actions, selection overhead, and peak memory usage. For derivable tasks, goal-reaching measurements stop when g is first derived, including for Chase. Chase additionally continues to the complete closure, whose cost and runtime are reported separately. For non-derivable tasks, measurements stop when deduction is exhausted. We report averages over tasks and independent runs, together with variability where applicable.

Parameters and implementation. Unless stated otherwise, we use $\alpha =$ **X**, $\beta =$ **X**, $\lambda =$ **X**, $\gamma =$ **X**, HDC dimension $D =$ **X**, randomized-policy temperature $\tau =$ **X**, and a fair fallback every **X** selections. The learned policy uses **X** training executions and **X** action-conditioned rollouts per sampled state. Experiments run on **X** CPUs, **X** GB RAM, and **X** GPUs. Each randomized experiment is repeated **X** times. We report mean values and **X** confidence intervals.

Experimental results. We next report our findings. We conduct six sets of experiments evaluating (Exp-1) overall goal-reaching efficiency, (Exp-2) effectiveness of goal-aware scoring, (Exp-3) perfor-

Method	FB15k-237			ogbl-biokg			FinReflectKG			DBpedia			Synthetic		
	Rec.	Time	#Exp.	Rec.	Time	#Exp.	Rec.	Time	#Exp.	Rec.	Time	#Exp.	Rec.	Time	#Exp.
Chase	X	742.46*(X ×)	5,019*(X %)	X	X	X	X	X	X	X	X	X	X	X	X
DepOnly	X	8.56 (X ×)	111.50(X %)	X	X	X	X	X	X	X	X	X	X	X	X
Deterministic	X	9.77(X ×)	104.89(X %)	X	X	X	X	X	X	X	X	X	X	X	X
Random	X	10.33(X ×)	103.11(X %)	X	X	X	X	X	X	X	X	X	X	X	X
Learned	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
Hybrid-NoLLM	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
AdaDeduce	X	6.62	9.72	X	X	X	X	X	X	X	X	X	X	X	X
MiniClean	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
HyperBlocker	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
AdaDeduce$_{GPU}$	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X

Table 2: Overall effectiveness per dataset: goal recall (Rec., fraction of derivable goals reached), end-to-end runtime (Time, seconds), and evaluated actions (#Exp., in thousands); fewer expansions relative to Chase are marked by ↓X %.

mance of inference policies, (Exp-4) adaptive execution and correctness, (Exp-5) generalization and scalability, and (Exp-6) component contributions and parameter sensitivity.

Exp-1: Overall effectiveness. We first test whether agent AdaDeduce reduces the computation needed to reach a goal compared with unrestricted and goal-directed deduction, and whether its selection overhead is justified by the resulting savings.

Goal-reaching execution cost. Table 2 summarizes the recall, runtime and evaluated actions of all methods on the five datasets. On the four real-world datasets, we compare the goal-reaching costs of Chase, DepOnly, Deterministic, Random, and Learned. All methods derive the same target goals, and their costs include successful and unsuccessful evaluations. As shown in Figure 7(a), AdaDeduce reduces execution cost by **X** % on average relative to **X**, the fastest baseline for each dataset. The largest improvement is on **X**, where its cost is **X**, compared with **X** for **X**. The improvement comes from prioritizing promising grounded actions instead of applying unnecessary rules.

Runtime and inference work. Figure 7(b) compares end-to-end runtime on the same workloads. AdaDeduce reduces runtime by **X** % on average and achieves a maximum speedup of **X** × on **X**. As shown in Figure 7(c), it evaluates **X** % fewer actions and incurs **X** % fewer failed evaluations than **X**, which is the strongest baseline. These reductions explain the runtime gains beyond differences in individual predicate-evaluation costs.

Selection overhead. We measure the time spent on generating, scoring, and scheduling candidates, separately from action evaluation. Figure 7(d) shows that policy-selection overhead accounts for **X** % of AdaDeduce's total runtime, while avoiding **X** % of the evaluation work performed by DepOnly. Thus, the additional control cost is outweighed by the inference computation it eliminates.

Effect of reasoning complexity. We vary the minimum derivation depth from **X** to **X** and the number of alternative derivations from **X** to **X** per goal. Figure **X** shows that AdaDeduce reduces execution cost by **X**–**X** % as these quantities increase. This tests whether adaptive action selection becomes more valuable when the agent must choose among multiple deduction paths.

Exp-2: Goal-aware scoring. We test the scoring signals of Section 4: symbolic goal relevance, grounded evidence and estimated evaluation cost. We also empirically validate Propositions 2 and 3.

! Add a set of 4 figures: fig-score

Safe goal-relevance filtering. To evaluate Proposition 2, we compare the derivable goals obtained with unrestricted chase and with actions satisfying $r(g, \mu) > 0$. We vary the number of GARs from **X** to **X** and the proportion of goal-irrelevant rules from **X** % to **X** %. Figure **X** (a) shows that relevance filtering removes **X** % of candidate actions while losing **X** derivable goals. This verifies the safe filtering property and quantifies its benefit for candidate reduction.

Accuracy of HDC evidence estimation. To examine Proposition 3, we compare the estimated support $\widehat{e}(S, \mu)$ with exact support $e(S, \mu)$. We vary the HDC dimension D from **X** to **X** and the number of witnessed facts $|W(S)|$ from **X** to **X**. Figure **X** (b) reports mean absolute error and 95th-percentile error. The mean error decreases from **X** to **X** as D increases. We also measure the maximum pairwise interference ϵ of encoded facts and verify whether the observed errors satisfy the bound of Proposition 3. The results characterize when compact evidence encoding is accurate enough to guide inference.

Evidence-ranking effectiveness. We compare candidate rankings based on exact evidence support, HDC-estimated support, and no evidence signal. Figure **X** (c) reports Spearman rank correlation with exact evidence rankings and the resulting goal-reaching execution costs. HDC achieves correlation **X** and reduces ranking overhead by **X** % relative to exact evidence computation, while increasing execution cost by at most **X** %. This tests whether the approximation preserves useful action preferences.

Evaluation-cost estimation. We compare $\widehat{c}_{\text{eval}}(S, \mu)$ with the actual remaining evaluation cost of selected actions. By varying the proportion of ML preconditions from **X** % to **X** %, we test heterogeneous evaluation costs. Figure **X** (d) shows a prediction error of **X** and rank correlation of **X**. Combining evidence support with cost estimates reduces execution cost by **X** % compared with evidence-only ranking, showing why both signals are needed.

Exp-3: Policy effectiveness. We compare Deterministic, Random, and Learned under the same AdaDeduce scheme. We also test whether learned cost-to-go provides better decisions than immediate action scoring.

! Add a set of 4 figures: fig-policies

Policy comparison. We evaluate Deterministic, Random, and Learned on identical tasks, using the same candidate management, precondition reuse, and fairness mechanism. Figure **X** (a) compares execution cost and runtime. Random tests whether score-based exploration improves on the greedy choices of Deterministic; Learned



Figure 7: Overall performance under varying budgets (columns: success rate, utility, cost and runtime; rows: FB15k-237, ogbl-biokg, FinReflectKG and DBpedia, top to bottom; placeholders to be replaced by plots)

tests whether predicting downstream cost improves on immediate scoring. Learned reduces cost by **X** % and **X** % relative to Deterministic and Random, respectively. Its selection overhead is **X** % of total execution time.

Cost-to-go prediction. We sample **X** states and compare candidate predictions $\widehat{Q}_\theta(S, g, \mu)$ against measured remaining execution costs from action-conditioned rollouts. We report mean absolute error, rank correlation, and top-1 action selection agreement with the lowest measured-cost candidate. Figure **X** (b) shows prediction error **X**, rank correlation **X**, and top-1 agreement **X** %. Compared with immediate scoring f, cost-to-go learning reduces the execution cost by **X** %. This tests its ability to capture downstream inference costs.

Approximation to optimal policies. To examine the practical difficulty of PSP (Theorem 1), we construct small instances with **X**–**X** grounded actions and compute the minimum expected goal-reaching cost by exhaustive state-space search. We compare this reference with AdaDeduce and the other policies. Figure **X** (c) reports the ratio of achieved cost to optimum. Learned is within **X** × of the optimum on average, whereas exact optimization becomes **X** × slower as the action count increases from **X** to **X**. This empirically illustrates the optimization trade-off motivating PSP, without treating proof cost as the objective.

Learning from limited experience. We vary training executions from **X** to **X** and action-conditioned rollouts per state from **X** to **X**. Figure **X** (d) shows that the learned policy reduces execution cost by **X** % using **X** training runs, with further improvement of **X** % as training data increase. We also report performance on states not covered by training rollouts to quantify the effect of limited experience.

Exp-4: Adaptive execution and correctness. We next evaluate evidence reuse, event-driven candidate reactivation, and fair execution. We also empirically check the soundness, goal-level completeness, and termination of AdaDeduce (Theorems 4–6).

! Add a set of 4 figures: fig-adaptation

Reusing failed evaluations. We construct tasks in which candidates initially fail because dynamic preconditions are missing and later become enabled. We vary the ratio of such candidates from **X** % to **X** %. Figure **X** (a) compares AdaDeduce with a variant that reevaluates all preconditions from scratch. Evidence reuse reduces repeated predicate checks by **X** % and total execution cost by **X** %, validating the practical value of recording unsuccessful evaluations in H.

Event-driven candidate management. We vary the number of suspended actions from **X** to **X** and the number of facts derived during execution. We compare incremental candidate updates with full candidate reconstruction after every deduction step. Figure **X** (b) shows that event-driven management reduces candidate-maintenance time by **X** % and redundant evaluations by **X** %. All dynamically enabled actions are reactivated when their missing facts are available.

Soundness and goal completeness. To validate Theorems 4 and 5, we compute exact chase closures on **X** tractable tasks, including **X** derivable and **X** non-derivable goals. For every execution, we verify that all derived facts belong to the chase closure and compare the goal result with the unrestricted chase. Table **X** reports **X** invalid deductions, **X** missed derivable goals, and **X** incorrectly established non-derivable goals. The results check the correctness guarantees independently of policy ranking accuracy.

! Add a table: tab-correctness

Termination. To evaluate Theorem 6, we generate recursive rule dependencies with **X**–**X** grounded actions and vary the number of dynamic failures. We measure termination rate, maximum reevaluations per action, and total evaluated actions. Figure **X** (c) reports **X** % termination and at most **X** reevaluations per action.

Fair execution. We also vary the fair-fallback interval from **X** to

X selections. Figure X (d) shows that all derivable goals remain reachable under fair execution, while the interval changes total cost by X–X %. This quantifies the efficiency cost of guaranteeing that low-ranked actions are not indefinitely ignored.

Exp-5: Generalization and scalability. We test whether learned policies transfer to unseen reasoning tasks and whether AdaDeduce remains efficient as graphs, rule libraries, and candidate sets grow.

! Add a set of 4 figures: fig-generalization

Generalization across workloads. We train the learned policy on X tasks from X and evaluate it on unseen goals, relation types, and rule dependencies. Figure X (a) compares execution cost against deterministic scoring and a policy trained on the test workload. The transferred policy reduces cost by X % over deterministic scoring, versus X % for in-domain training. This shows how well execution experience generalizes beyond previously observed deductions.

Graph scalability. We vary synthetic graph size from X to 100M vertices and from X to 1B edges, holding rule complexity fixed. Figure X (b) compares runtime and peak memory. Agent AdaDeduce processes the largest graph in X seconds using X GB memory, with runtime growing by X × as graph size increases by X ×. We separately report matching, candidate generation, scoring, and evaluation time to identify the dominant scalability costs.

Rule and candidate scalability. We vary the number of GARs from X to X and the number of grounded candidates from X to X. Figure X (c) shows that agent AdaDeduce reduces the number of evaluated actions by X % relative to DepOnly, while selection overhead increases from X to X. This tests whether the savings from selective deduction persist as the action space grows.

Expensive preconditions. We increase the fraction of ML preconditions from X % to X % and vary their evaluation costs by a factor of X. As Figure X (d) shows, cost-aware selection reduces total execution cost by X–X % over cost-oblivious selection, showing the benefit of policy-guided deduction for heterogeneous evaluation costs.

Exp-6: Ablation & sensitivity. We asses the contributions of symbolic relevance, HDC evidence, cost estimation, learning and adaptive execution, and test the parameters affecting their efficiency.

! Add a set of 4 figure: fig-ablation

Scoring components. We compare full AdaDeduce with three variants that remove goal relevance, evidence support, or evaluation-cost estimation, one at a time. Figure X (a) reports increases of X %, X %, and X % in execution cost, respectively. This identifies which scoring signals contribute most to efficient inference.

Evidence approximation and reuse. We compare three variants: exact evidence overlap, HDC-estimated overlap, and selection without evidence support. Figure X (b) shows that HDC reduces scoring time by X % relative to exact overlap, while increasing execution cost by only X %. Removing evidence support increases execution cost by X %. We separately disable cached precondition outcomes to distinguish approximate ranking from exact evaluation reuse.

Policy learning and adaptation. We remove cost-to-go learning, event-driven reactivation, and incremental candidate maintenance one at a time, retaining a fair execution mechanism. Figure X (c) reports execution-cost increases of X %, X % and X %, respectively. For reactivation, we compare against a correct full-rescanning alternative; a version that permanently discards dynamic failures is used only to illustrate lost completeness.

Parameter sensitivity. We vary the scoring weights α, β, λ, decay γ, HDC dimension D, randomized temperature τ, and fair-fallback interval. Figure X (d) shows that execution cost varies by at most X % within the tested parameter ranges. We report the configurations that minimize end-to-end cost, including policy-selection overhead, rather than optimizing scoring accuracy alone.

Summary. We find the following. (1) AdaDeduce reduces goal-reaching execution cost by X % and runtime by X % compared with the strongest baseline; (2) goal relevance safely eliminates X % of candidates, while HDC provides accurate evidence estimates with X % lower scoring overhead; (3) Learned selects actions using predicted cost-to-go to reduce unnecessary evaluations by X %; (4) adaptive execution reuses evidence and preserves soundness, completeness, and termination in the tested workloads; and (5) the benefits extend to unseen workloads and graphs with up to 100M vertices and 1B edges.

7 RELATED WORK

This section reviews prior work on deductive reasoning, query optimization, search policies and learned graph reasoning.

! Expand the section with recent work and citations

Goal-directed rule evaluation. Magic sets restrict rule evaluation to query-relevant facts [9], while QSQ and tabling use top-down evaluation and memoization [50, 55]. Chase-based systems derive logical consequences of rules [2, 10], with scalable implementations such as RDFox and Vadalog [11, 42]. These approaches improve deduction by restricting or accelerating rule evaluation.

In contrast, AdaDeduce further optimizes the order of individual grounded rule applications according to their expected contribution and execution cost, stopping once the target goal is derived.

Incremental deduction and truth maintenance. Semi-naive evaluation avoids redundant rule applications by propagating newly derived facts [8], while incremental Datalog evaluation and Delete-and-Rederive (DRed) maintain deductions under data updates [28]. These techniques reduce redundant computation during deductive evaluation. AdaDeduce similarly exploits incremental updates, but additionally maintains action-level execution evidence, suspends applications with missing dynamic preconditions, and reactivates them as new facts become available. Its distinctive objective is to minimize the computation spent on reaching a specific goal under a learned or heuristic selection policy.

Cost-based and learned query optimization. Classical query optimizers choose efficient execution plans using cost estimates [26, 47]. Adaptive query processing, such as Eddies [4], revises execution decisions as runtime conditions change. Learned optimizers, including Neo and Bao [37, 38], exploit execution experience to improve plan selection. These techniques primarily optimize query operators or execution plans. In contrast, AdaDeduce optimizes grounded inference actions that may derive new facts, enable subsequent deductions, or reveal reusable evidence even when unsuccessful. Its

learned cost-to-go estimates the remaining computation needed to reach a logical goal, not the cost of executing a fixed query plan.

Policy-guided reasoning and search. A* and AND/OR search prioritize alternatives using cost estimates [29, 39]. Monte Carlo tree search balances exploration and exploitation [12], while metareasoning explicitly accounts for computational effort [15, 45, 61]. Meta-Chase [22] introduces policy-driven graph deduction through adaptive rule-set selection. Our work differs by selecting individual grounded rule applications rather than rule sets or abstract search states. Moreover, the selection objective includes failed evaluations and policy overhead, while fair execution preserves goal-level completeness without requiring accurate cost estimates.

Learned and neuro-symbolic graph reasoning. Neural reasoners learn representations of paths and subgraphs to predict missing facts [51, 59, 60]. Rule-learning systems discover weighted logical patterns [40, 41, 44], while reinforcement-learning agents explore knowledge graphs through sequential decisions [14, 56]. LLM-based graph agents further use language models to guide graph exploration [36, 49, 57]. These methods primarily learn rules, predict answers, or select reasoning paths. In contrast, AdaDeduce uses learning to control the execution of existing GARs. Every derived fact requires exact precondition validation, so learned predictions affect efficiency without changing logical soundness.

Approximate evidence representation and provenance. Hyperdimensional computing (HDC) represents structured information using high-dimensional vectors and supports efficient approximate similarity computation [33]. We exploit this representation to estimate the overlap between grounded preconditions and accumulated evidence, with an explicit approximation bound. Provenance semirings represent the derivations supporting query answers [27], while ranked enumeration and top-k Datalog techniques order answers or derivations by cost [16, 54]. Unlike these approaches, which describe or rank derivations, AdaDeduce minimizes the computation spent on discovering a goal-reaching derivation, including unsuccessful attempts and adaptive selection overhead.

Positioning of our work. AdaDeduce bridges deductive reasoning, adaptive query optimization, and learned inference control. Unlike goal-directed deduction, which restricts relevant rules, or query optimization, which schedules predefined operators, AdaDeduce adaptively selects grounded inference actions whose outcomes change subsequent deduction opportunities. By combining symbolic relevance, reusable execution evidence, and learned cost-to-go, it minimizes actual goal-reaching computation while preserving soundness, goal-level completeness, and termination.

8 CONCLUSION

We have presented policy-guided deduction, a framework for agentized graph reasoning that shifts optimization from completed proofs to the actual cost of reaching a target goal. We have formalized the policy selection problem and established its intractability. To address it, we have developed goal-aware scoring that combines symbolic dependencies, reusable execution evidence, and evaluation-cost estimates, together with learned cost-to-go policies. Our AdaDeduce gent integrates these techniques through adaptive action selection and incremental deduction, while guaranteeing soundness, goal-level completeness under fair execution, and termination. The results establish a foundation for efficient, verifiable, and cost-aware agent-controlled logical reasoning. We have experimentally verified that AdaDeduce is promising in practice.

Future work includes (1) extending policy-guided deduction to evolving graphs and rules; (2) learning transferable policies for unseen reasoning tasks; and (3) coordinating multiple inference agents for efficient collaborative and complex reasoning tasks.

REFERENCES

[1] 2026. Full version. *link*.

[2] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.

[3] Abhinav Arun, Fabrizio Dimino, Tejas Prakash Agarwal, Bhaskarjit Sarmah, and Stefano Pasquali. 2025. FinReflectKG: Agentic Construction and Evaluation of Financial Knowledge Graphs. *arXiv preprint arXiv:2508.17906* (2025).

[4] Ron Avnur and Joseph M. Hellerstein. 2000. Eddies: Continuously Adaptive Query Processing. In *SIGMOD*. ACM, 261–272.

[5] Guillaume Bagan, Angela Bonifati, Radu Ciucanu, George H. L. Fletcher, Aurélien Lemay, and Nicky Advokaat. 2017. gMark: Schema-Driven Generation of Graphs and Queries. *IEEE Trans. Knowl. Data Eng.* 29, 4 (2017), 856–869.

[6] Wenchao Bai, Wenfei Fan, Daji Li, Jian Li, Shuhao Liu, Mingliang Ouyang, and Qiang Yuan. 2025. MiniClean: A Single-Machine System for Cleaning Big Graphs. In *SIGMOD (demo)*.

[7] Wenchao Bai, Wenfei Fan, Shuhao Liu, Kehan Pang, Xiaoke Zhu, and Jiahui Jin. 2025. Rule-Based Graph Cleaning with GPUs on a Single Machine. In *SIGMOD*.

[8] François Bancilhon and Raghu Ramakrishnan. 1986. An Amateur's Introduction to Recursive Query Processing Strategies. In *SIGMOD*. ACM Press, 16–52.

[9] Catriel Beeri and Raghu Ramakrishnan. 1991. On the Power of Magic. *J. Logic Programming* 10, 3–4 (1991), 255–299.

[10] C. Beeri and M.Y. Vardi. 1984. A proof procedure for data dependencies. *JACM* 31, 4 (1984), 718–741.

[11] Luigi Bellomarini, Emanuel Sallinger, and Georg Gottlob. 2018. The Vadalog System: Datalog-based Reasoning for Knowledge Graphs. *PVLDB* 11, 9 (2018), 975–987.

[12] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. 2012. A Survey of Monte Carlo Tree Search Methods. *IEEE Trans. Comput. Intell. AI Games* 4, 1 (2012), 1–43.

[13] Weidong Chen and David Scott Warren. 1996. Tabled Evaluation With Delaying for General Logic Programs. *J. ACM* 43, 1 (1996), 20–74.

[14] Rajarshi Das, Shehzaad Dhuliawala, Manzil Zaheer, Luke Vilnis, Ishan Durugkar, Akshay Krishnamurthy, Alex Smola, and Andrew McCallum. 2018. Go for a Walk and Arrive at the Answer: Reasoning over Paths in Knowledge Bases Using Reinforcement Learning. In *ICLR*.

[15] Thomas Dean and Mark Boddy. 1988. An Analysis of Time-Dependent Planning. In *AAAI*. 49–54.

[16] Daniel Deutch, Amir Gilad, and Yuval Moskovitch. 2015. Selective Provenance for Datalog Programs Using Top-k Queries. *PVLDB* 8, 12 (2015), 1394–1405.

[17] Wenfei Fan, Lihang Fan, Dandan Lin, and Min Xie. 2025. Explaining GNN-based Recommendations in Logic. *PVLDB* 18, 3 (2025), 715–728.

[18] Wenfei Fan, Wenzhi Fu, Ruochun Jin, Ping Lu, and Chao Tian. 2022. Discovering association rules from big graphs. *PVLDB* 15, 7 (2022), 1479–1492.

[19] Wenfei Fan, Ruochun Jin, Muyang Liu, Ping Lu, Chao Tian, and Jingren Zhou. 2020. Capturing Associations in Graphs. *PVLDB* 13, 11 (2020), 1863–1876.

[20] Wenfei Fan, Yang Leng, Daji Li, Shuhao Liu, Mingliang Ouyang, Yaoshu Wang, Min Xie, and Qiang Yuan. 2025. DreamCreek: AI for Battery Formation and Grading. In *ICDE*. IEEE, 4544–4547.

[21] Wenfei Fan, Daji Li, Peiyu Liang, Shuhao Liu, Yaoshu Wang, Yiming Wang, Min Xie, and Runjie Zhang. 2024. Graph Association Analyses for Early Drug Discovery. *PVLDB* 17, 12 (2024), 4293–4296.

[22] Wenfei Fan, Xiaomeng Lou, and Ping Lu. 2027. From Chase to Meta Chase. In *ICDE*. IEEE.

[23] Michael Garey and David Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company.

[24] Malik Ghallab, Dana S. Nau, and Paolo Traverso. 2004. *Automated planning - theory and practice*. Elsevier.

[25] Goetz Graefe. 1995. The Cascades Framework for Query Optimization. *IEEE Data Eng. Bull.* 18, 3 (1995), 19–29.

[26] Goetz Graefe and William J. McKenna. 1993. The Volcano Optimizer Generator: Extensibility and Efficient Search. In *ICDE*. IEEE Computer Society, 209–218.

[27] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. 2007. Provenance Semirings. In *PODS*. 31–40.

[28] Ashish Gupta, Inderpal Singh Mumick, and V. S. Subrahmanian. 1993. Maintaining Views Incrementally. In *SIGMOD*. ACM Press, 157–166.

[29] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. 1968. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Trans. Syst. Sci. Cybern.* 4, 2 (1968), 100–107.

[30] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. In *NeurIPS*.

[31] Pentti Kanerva. 2009. Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors. *Cognitive Computation* 1, 2 (2009), 139–159.

[32] Samir Khuller, Anna Moss, and Joseph Naor. 1999. The budgeted maximum coverage problem. *Inf. Process. Lett.* 70, 1 (1999), 39–45.

[33] Denis Kleyko, Dmitri A. Rachkovskij, Evgeny Osipov, and Abbas Rahimi. 2023. A Survey on Hyperdimensional Computing aka Vector Symbolic Architectures, Part I: Models and Data Transformations. *ACM Comput. Surv.* 55, 6 (2023), 130:1–130:40.

[34] Donald E. Knuth. 1977. A generalization of Dijkstra's algorithm. *Inf. Process. Lett.* 6, 1 (1977), 1–5.

[35] Jens Lehmann, Robert Isele, Max Jakob, Anja Jentzsch, Dimitris Kontokostas, Pablo N. Mendes, Sebastian Hellmann, Mohamed Morsey, Patrick van Kleef, Sören Auer, and Christian Bizer. 2015. DBpedia - A large-scale, multilingual knowledge base extracted from Wikipedia. *Semantic Web* 6, 2 (2015), 167–195.

[36] Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. 2024. Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning. In *ICLR*.

[37] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making Learned Query Optimization Practical. In *SIGMOD*. ACM, 1275–1288.

[38] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, and Nesime Tatbul. 2019. Neo: A Learned Query Optimizer. *PVLDB* 12, 11 (2019), 1705–1718.

[39] Alberto Martelli and Ugo Montanari. 1973. Additive AND/OR Graphs. In *IJCAI*. 1–11.

[40] Christian Meilicke, Melisachew Wudage Chekol, Daniel Ruffinelli, and Heiner Stuckenschmidt. 2019. Anytime Bottom-Up Rule Learning for Knowledge Graph Completion. In *IJCAI*. ijcai.org, 3137–3143.

[41] Christian Meilicke, Manuel Fink, Yanjie Wang, Daniel Ruffinelli, Rainer Gemulla, and Heiner Stuckenschmidt. 2018. Fine-Grained Evaluation of Rule- and Embedding-Based Systems for Knowledge Graph Completion. In *ISWC*. 3–20.

[42] Yavor Nenov, Robert Piro, Boris Motik, Ian Horrocks, Zhe Wu, and Jay Banerjee. 2015. RDFox: A Highly-Scalable RDF Store. In *ISWC*. 3–20.

[43] Kehan Pang, Wenfei Fan, Min Xie, and Dandan Lin. 2026. Explaining GNN Negatives Globally and Locally. In *ICDE*.

[44] Meng Qu, Junkun Chen, Louis-Pascal Xhonneux, Yoshua Bengio, and Jian Tang. 2020. RNNLogic: Learning logic rules for reasoning on knowledge graphs. *arXiv preprint arXiv:2010.04029* (2020).

[45] Stuart Russell and Eric Wefald. 1991. Principles of Metareasoning. *Artificial Intelligence* 49, 1–3 (1991), 361–395.

[46] Fereidoon Sadri and Jeffrey D. Ullman. 1980. The Interaction between Functional Dependencies and Template Dependencies. In *SIGMOD*.

[47] Patricia G. Selinger, Morton M. Astrahan, Donald D. Chamberlin, Raymond A. Lorie, and Thomas G. Price. 1979. Access Path Selection in a Relational Database Management System. In *SIGMOD*. ACM, 23–34.

[48] Shenzhen Institute of Computing Sciences. 2022. Fishing Fort. *https://en.sics.ac.cn/col84/index*.

[49] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. 2024. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. In *ICLR*.

[50] Terrance Swift and David S. Warren. 2012. XSB: Extending Prolog with Tabled Logic Programming. *Theory Pract. Log. Program.* 12, 1–2 (2012), 157–187.

[51] Komal K. Teru, Etienne G. Denis, and William L. Hamilton. 2020. Inductive Relation Prediction by Subgraph Reasoning. In *ICML*. 9448–9457.

[52] Kristina Toutanova, Danqi Chen, Patrick Pantel, Hoifung Poon, Pallavi Choudhury, and Michael Gamon. 2015. Representing text for joint embedding of text and knowledge bases. In *EMNLP*.

[53] TypeSafe AI. 2026. Introducing System One Models & Jev. https://typesafe.ai/blog/introducing-system-one-models-and-jev.

[54] Nikolaos Tziavelis, Wolfgang Gatterbauer, and Mirek Riedewald. 2021. Beyond Equi-Joins: Ranking, Enumeration and Factorization. *PVLDB* 14, 11 (2021), 2599–2612.

[55] Laurent Vieille. 1986. Recursive Axioms in Deductive Databases: The Query/-Subquery Approach. In *Expert Database Conf.* 253–267.

[56] Wenhao Xiong, Thien Hoang, and William Yang Wang. 2017. DeepPath: A Reinforcement Learning Method for Knowledge Graph Reasoning. In *EMNLP*.

[57] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *ICLR*.

[58] Xiaoke Zhu, Min Xie, Ting Deng, and Qi Zhang. 2025. HyperBlocker: Accelerating Rule-based Blocking in Entity Resolution Using GPUs. *PVLDB* 18, 2 (2025).

[59] Zhaocheng Zhu, Xinyu Yuan, Mikhail Galkin, Sophie Xhonneux, Ming Zhang, Maxime Gazeau, and Jian Tang. 2023. A*Net: A Scalable Path-based Reasoning Approach for Knowledge Graphs. In *NeurIPS*. 59323–59336.

[60] Zhaocheng Zhu, Zuobai Zhang, Louis-Pascal Xhonneux, and Jian Tang. 2021. Neural Bellman-Ford Networks: A General Graph Neural Network Framework for Link Prediction. In *NeurIPS*. 29476–29490.

[61] Shlomo Zilberstein. 1996. Using Anytime Algorithms in Intelligent Systems. *AI Magazine* 17, 3 (1996), 73–83.

A PROOF OF THEOREM 1

We first show that both problems admit polynomial-size certificates that can be verified in polynomial time (PTIME). A certificate is a reasoning chain $C = \langle(\varphi_1, h_1), \ldots, (\varphi_k, h_k)\rangle$. Its length is bounded by the number of distinct motifs, not by B: a failed step or a repeated application can be dropped from a certificate, which still derives g at no larger cost, so a shortest certificate applies each motif at most once, and k is polynomial in $|G|$ and $|\Sigma|$.

Upper bounds. We guess a reasoning chain starting from Γ_0. At each step, we check that: (a) φ_i is applicable under h_i *w.r.t.* the current fact set (predicate evaluation is polynomial in the size of G and Γ), and (b) the derived fact is correctly added. This yields Γ_k (or $\Gamma_{\le B}$) in $O(k \cdot T_{\text{apply}})$ time. Then we verify the following.

- OPSM: We verify that $g \in \Gamma_k$ and $\text{cost}(C) \le B$. Since k is polynomial in $|G|$ and $|\Sigma|$ and each step is polynomial, verification is polynomial.
- BOSP. We additionally compute $U(\Gamma_{\le B}, g)$; U is a weighted coverage of subgoal instances, computable in polynomial time from $\Gamma_{\le B}$. We then check $U(\Gamma_{\le B}, g) \ge \eta$ and $\text{cost}(C) \le B$.

Thus, both problems are in NP.

Lower bounds. We reduce from classical NP-complete problems, building one gadget for both.

! Add proofs for if and only directions!

OPSM. We reduce from Set Cover [23], under unit cost: every application, successful or not, costs 1. Given a universe $\mathcal{U} = \{e_1, \ldots, e_n\}$, sets $\mathcal{S} = \{S_1, \ldots, S_m\}$ and an integer K, build a graph with a vertex s_i per set and a vertex u_j per element, an edge $\text{member}(s_i, u_j)$ whenever $e_j \in S_i$, and an initial fact $\text{avail}(s_i)$ per set. The rules are: (a) $\text{avail}(x) \to \text{on}(x)$, activating a set; (b) for each $e_j \in S_i$, $\text{member}(x, y) \wedge \text{on}(x) \to \text{cov}(y)$, covering an element of an active set; and (c) $\bigwedge_{j=1}^{n} \text{cov}(y_j) \wedge \bigwedge_{i<j}(y_i \neq y_j) \to g$, with a pairwise precondition $y_i \neq y_j$ per pair, in the idiom of φ_4 of Example 2: matches are homomorphisms, and without them one covered element could serve as every y_j. A chain derives g iff it activates a cover of $\mathcal{U}$, at a cost of $b + n + 1$ for b activated sets, n coverings and the final application; hence a chain of cost at most $K + n + 1$ exists iff a cover of size at most K does.

BOSP. The same gadget reduces from budgeted maximum coverage [32], which asks for at most K sets covering at least η elements: declare each covered element $\text{cov}(u_j)$ a subgoal of weight 1, and set the budget to $K + \eta$ and the threshold to η. A chain of cost at most $K + \eta$ and utility at least η activates b sets and covers $c \ge \eta$ elements at cost $b + c$, so $b \le K$ and the b active sets cover at least η elements; conversely, such sets yield such a chain. The sequential structure is respected: a set must be activated before its elements can be covered.

Both constructions are in PTIME, and solutions correspond to valid reasoning chains. Hence OPSM and BOSP are NP-hard, and NP-complete with the upper bounds. The hardness is in the combined complexity, with the rules part of the input, and rests on shared subderivations being charged once: charged as trees, with shared premises paid anew, the minimum would be computable in polynomial time by a tropical fixpoint [34]. □

B PROOF OF THEOREM 4

The agent may attempt a candidate $\mu = (\varphi, h)$ with untested preconditions, but only a successful application derives: one whose preconditions all hold under h *w.r.t.* the current fact set is a chase step, deriving a fact logically implied by existing facts and Σ, and a failed one adds nothing. Since the chase computes the closure of all such valid applications, every fact derived by the policy must also appear in Γ^*. Thus, policy-guided deduction derives only entailed facts. □

C PROOF OF THEOREM 5

Since $g \in \Gamma^*$, there exists a chase sequence that derives g from Γ_0. We construct a policy that, at each step, selects the next rule application in this sequence. This policy induces a reasoning chain that exactly follows the chase derivation and reaches g. Thus, any entailed goal can be derived by a suitable policy.

Under fairness, at every state, every motif whose preconditions hold is eventually tried: a motif failing on static preconditions never has them hold later and stays retired, and one failing on dynamic preconditions is re-admitted as Γ grows, so no motif of a valid derivation of g is ignored when its turn comes. Since the chase sequence that derives g consists of finitely many such motifs, the policy will eventually apply all of them. Thus, g is derived in finite steps. □

D PROOF OF THEOREM ??

Write U_i for the utility gathered after i steps of the pass, Γ_i for the fact set, and $U^* = U(C^*)$; let O be the at most B' motifs of C^*, each available to the pass by hypothesis. By monotonicity and submodularity,

$$U^* \le U_i + \textstyle\sum_{\mu \in O} \Delta U(\mu \mid \Gamma_i) \le U_i + B' \cdot \max_\mu \Delta U(\mu \mid \Gamma_i),$$

so some available motif adds at least $(U^* - U_i)/B'$, and the chosen one loses at most ϵ against it: $U^* - U_{i+1} \le (1 - 1/B')(U^* - U_i) + \epsilon$. Unfolding the recurrence over B' steps, $U^* - U_{B'} \le (1 - 1/B')^{B'} U^* + B'\epsilon \le U^*/e + B'\epsilon$. □

E PROOF OF THEOREM ??

Call a chain *irredundant* when every step in it derives a fact that is used, as a premise of a later step or as g; an optimal chain is irredundant, since dropping a step that failed or whose fact is never used still derives g at strictly lower cost. Along an irredundant chain, the suffix that begins with a candidate μ projects to a walk from its consequent p_0 to g in $D_{\Sigma,g}$; with edges weighted by w_0, the suffix costs at least $\text{price}(\mu) = w_0(\varphi) + d(p_0)$. A branch is cut only when the cost so far plus this bound exceeds B or reaches $\hat{C}$, so every irredundant chain through a cut branch costs more than B or no less than $\hat{C}$; and a redundant chain is dominated by its irredundant subchain, which the sibling branches explore at strictly smaller cost, each skipped step unpaid. A dynamic failure bars a motif only at the fact set that saw it fail, so no chain is lost to a stale record. With no top-k truncation nothing else is discarded, a chain is recorded only within B, and the search ends only when every branch is cut or completed; hence a chain is recorded whenever one fits B, and no cheaper chain to g within B is missed. □

Algorithm BuildDG
Input: goal g, rule set Σ.
Output: $D_{\Sigma,g} = (N, A)$ and table d.

1. $N \leftarrow \{\text{pred}(g)\}$, $A \leftarrow \emptyset$, $d[\text{pred}(g)] \leftarrow 0$; enqueue $\text{pred}(g)$ into queue Q;
2. **while** Q is not empty **do**
3. dequeue p from Q;
4. **for each** rule $\varphi \in \Sigma$ with consequent p **do**
5. **for each** predicate q in the pattern or preconditions of φ **do**
6. add $q \to p$ to A;
7. **if** $q \notin N$ **then** $N \leftarrow N \cup \{q\}$, $d[q] \leftarrow d[p] + 1$, enqueue q into Q;
8. **return** $D_{\Sigma,g} = (N, A)$ and d.

Figure 8: Building $D_{\Sigma,g}$ and $d(\cdot)$.

Algorithm ScoreF
Input: $D_{\Sigma,g} = (N, A)$ with table d, rule φ of the candidate, decay factor $\gamma \in (0, 1)$.
Output: $\text{rel}_{\text{fam}}(\mu)$.

1. $s \leftarrow 0$;
2. **for each** predicate q of φ (pattern, preconditions and consequent) **do**
3. **if** $q \notin N$ **then return** 0; /* *outside $D_{\Sigma,g}$: filtered out* */
4. $s \leftarrow s + d[q]$;
5. **return** γ^s.

Figure 9: Scoring a candidate from $D_{\Sigma,g}$.

F PROOF OF THEOREM 6

One can verify that the number of distinct ground facts that can be derived is finite (bounded by $|V|$, $|\Sigma|$, and the attribute domains). So is the number of candidate motifs, the matches of Σ in G. Each step either adds a new fact to Γ, no fact added twice, or retires a candidate, recording its failure and never retrying it at the same state; a dynamic failure is re-admitted only when Γ has grown, so each motif is tried at most once per distinct fact set, at most $|\Gamma^*| + 1$ sets in all. Both pools are finite, hence the reasoning process performs finitely many steps, at most the number of motifs times $|\Gamma^*| + 1$, and must terminate. □

G BOUNDED COST AND CONVERGENCE

Theorem 7: *Let $m = \max_S |\mathcal{A}(S)|$ be the largest candidate set, T_{score} the cost of scoring one candidate, and T_{eval} the cost of one application. Then the total runtime of policy-guided deduction over B steps is $O\big(B \cdot (m \cdot T_{score} + T_{eval})\big)$.* □

Proof sketch: Each step scores the at most m candidates in $O(m \cdot T_{\text{score}})$ and evaluates the selected one in T_{eval}. Over B steps, this yields $O(B \cdot (m \cdot T_{\text{score}} + T_{\text{eval}}))$. Matching maintenance is control overhead: it adds to the wall-clock beyond this bound, never to the budget. □

Theorem 8: *If execution continues without a budget constraint and the policy eventually explores all applicable motifs (i.e., is fair), then the resulting fact set converges to the chase closure Γ^*.* □

Proof sketch: By fairness, every applicable motif is eventually applied, hence all facts in Γ^* are derived. Since derivations are sound, no extra facts are produced, so the limit equals Γ^*. Termination implies convergence to this fixed point. □

H ALGORITHMS FOR THE DEPENDENCY GRAPH

Algorithm BuildDG (Figure 8) constructs the dependency graph $D_{\Sigma,g}$ and the distance table $d(\cdot)$, by a backward BFS from the goal over the rules; when rule edges are weighted, the BFS becomes a Dijkstra computation, relaxing $d[q] \leftarrow \min(d[q], d[p] + w_0(\varphi))$ at line 7, where $w_0(\varphi)$ is the certified lower bound on the cost of one application of φ, at least 1. It visits each rule once per predicate of the rule, *i.e.*, in $O(|\Sigma|)$ time. The table also bounds completion: any chain through a candidate with consequent p_0 completes at cost at least $d(p_0)$, since each application on the way costs at least its edge weight and the path is shortest. The completion price of a candidate is $\text{price}(\mu) = w_0(\varphi) + d(p_0)$: its own application, certified below by w_0, plus the cheapest completion thereafter. Algorithm ScoreF (Figure 9) scores a candidate motif $\mu = (\varphi, h)$ from the table. The match h plays no role here: rel_{fam} is a property of the rule, and the match enters only in rel_{evd}.

The learned policies read the graph through a vector $e_D = \bigoplus_k \rho^k(b_k)$, where b_k bundles the vectors of the predicates k rule applications from g; shifted onto their own channels, with each predicate entering the channel of its nearest depth, the shifts interfere only through their quasi-orthogonality, priced in Appendix I, and a predicate that cannot reach g never enters e_D. The vector is settled in the same backward pass as $d(\cdot)$, once per goal; for Example 7, $e_D = v_{\text{HighRisk}} \oplus \rho(b_1)$, with b_1 bundling owns, transfers, amount $> \theta$, M_{anom} and uses. A candidate bundles the predicates of its rule, premises and consequent alike, each once, into $v_\varphi = \bigoplus_{q \in \varphi} v_q$ and probes e_D per depth: the cosine $s_k = \cos(\rho^k(v_\varphi), e_D)$ reads how many of them sit k applications from g, up to the error of Appendix I, the shift ρ^k singling out the depth-k part of e_D. The probes give a profile $(s_0, s_1, \dots)$ per candidate, *e.g.*, proportional to $(s_0, s_1) = (1, 4), (1, 3), (1, 1)$ for candidates of $\varphi_1, \varphi_2, \varphi_3$ of Example 7, and $(0, 1)$ for φ_4, whose other premises sit on no channel. The profile is fed whole to f_θ, a fixed-width encoding that generalizes across predicates and goals, where the handcrafted rel_{fam} reads the same distances exactly, from the table (Figure 9), *e.g.*, the $\gamma^4, \gamma^3, \gamma$ of Example 7.

I ERROR OF COSINE READINGS

We justify the counts read from cosines in Section ??: the reading error is worst-case, data-free, and shrinks as the dimension grows, apart from an explicit collision bias. Each symbol carries a vector of D independent coordinates, uniform in $\{-1/\sqrt{D}, +1/\sqrt{D}\}$, hence of unit norm, and distinct symbols are independent; the vectors are generated on the fly from identifiers by hashing, and nothing is materialized per vertex; fact vectors are normalized to unit length, and each predicate enters the dependency encoding at its nearest depth only.

Lemma 9: *Let the probe u bundle m premise vectors and E bundle n fact vectors, all of unit length, let c count the premise facts occurring in E, let s count spurious symbol collisions — pairs of distinct facts, one in each bundle, that share a predicate or a vertex in the same role position without sharing the fact — and write $\tilde{s} = \sum 1/(1+a)$ over the same pairs, a the arity of the colliding facts. For $m + n \le \sqrt{D}$, with probability at least $1 - \delta$ over the random vectors, $|\cos(u, E) - (c + \tilde{s})/\sqrt{mn}| \le \sqrt{2\ln(2/\delta)/D}\,(1 + o(1))$.* □

Proof sketch. Write $\cos(u, E) = \langle u, E\rangle/(\|u\|\,\|E\|)$. The inner product splits into a signal c from the shared facts, a collision term $\tilde{s}$, and a noise of independent terms of $\pm 1/D$, at most mnD of them: a pair of distinct arity-a facts sharing a role-position symbol contributes exactly $1/(1+a)$, deterministically, the shared symbol carrying that weight in each unit-length fact vector. By Hoeffding the noise exceeds $\sqrt{2mn\ln(2/\delta)/D}$ with probability at most δ. The norms concentrate as $\|u\| = \sqrt{m}\,(1 + o(1))$ and $\|E\| = \sqrt{n}\,(1 + o(1))$ for $m + n \le \sqrt{D}$. Dividing yields the bound, which is free of m and n. When no predicate-role or vertex-role pair repeats across the bundles, $s = 0$ and the reading recovers c; realistic chains may repeat predicates, and the reading then over-counts by $\tilde{s}/\sqrt{mn}$, at most $s/\sqrt{mn}$, a bias, not noise. □

The bound prices both readings of Section **??**. For the evidence score, m is the few premises of a candidate and n the facts of the chain: adjacent counts differ by $1/\sqrt{mn}$, against a noise of $\sqrt{2\ln(2/\delta)/D}$, *e.g.*, a third of that step at $m = 1$, $n = 100$, $D = 10^4$ and $\delta = 0.01$; an absent premise has $c = 0$, hence $|\cos| \le 0.033$ when no role repeats; the bias stays small when repeated roles within a bundle are few, and when a chain repeats one predicate in the same role, *e.g.*, long HighRisk propagations, s grows like mn and the reading over-counts systematically, to be quantified in the experiments. The depth probes s_k read the same way: the dependency encoding bundles N predicates on disjoint channels, and the cosine reads $c_k/\sqrt{mN}$ against the same noise, reliable while $m + N$ stays below $\sqrt{D}$, so D grows with $(m+N)^2$ for larger libraries. The readings are worst-case over the random vectors and need no data; a learned embedding reads nothing of what its scores count.

J LLM-GENERATED SELECTION GUIDES

This section specifies the selection guide of Section 5.1 for new rules or domains with limited execution experience. The LLM supplies a semantic prior before deduction; the deduction engine makes all subsequent decisions without further LLM calls.

Input preparation. The engine first constructs $D_{\Sigma,g}$ and applies the exact relevance filter of Proposition 2. It then provides the LLM with: (a) the ground goal g, including its predicate and argument identifiers; (b) each retained rule, with a stable identifier, its pattern, preconditions, consequent, and available descriptions of predicate and ML-model meanings; and (c) available statistics of G, such as vertex-label and edge-label counts, degree distributions, and attribute summaries. Goal-local statistics may be included when available; unavailable statistics are explicitly marked as unknown. The prompt contains these summaries rather than the entire graph, and requires neither prior execution logs nor completed training runs. Domain knowledge comes from the LLM and the supplied symbol descriptions; opaque identifiers alone need not provide enough semantic information for a useful guide.

Output contract. The LLM returns a structured record $(id_\varphi, b_\varphi, reason_\varphi)$ for each retained rule φ, where $b_\varphi \in [-1, 1]$ is an initial preference and $reason_\varphi$ briefly explains the domain assumption supporting it. Positive values favor earlier consideration, negative values defer it, and zero denotes no preference. The prompt asks which domain mechanisms are promising for the given goal, conditional on the supplied rules and graph summaries. It explicitly permits abstention by returning zero. The explanation is retained for inspection, not used as a logical premise. The guide contains no new rule, inferred fact, hard exclusion, or numerical prediction of execution time or predicate truth. Dependency reachability, witnessed evidence, and measured evaluation costs remain the responsibility of the existing mechanisms.

Validation. The engine checks rule identifiers, record uniqueness, and the numeric range of every preference. Unknown identifiers and malformed records are ignored; missing entries receive $b_\varphi = 0$. If the request fails or the response cannot be parsed, all preferences default to zero and deduction proceeds with the original policy. No repair call is required during reasoning.

Using the guide. The scheduling level matters. Algorithm 6 expands one star and drains its ready set before expanding the next. Consequently, adding the same b_φ to every action of that star cannot distinguish its matches. For this algorithm, the guide therefore controls the choice of the next unexpanded star at line 2, while the selected policy continues to rank actions within the ready set using f or $\widehat{Q}_\theta$ without modification.

A concrete integration uses a fixed baseline star order, identical in the guided and unguided runs. For $m > 1$ retained stars, let $a(\varphi) = (m - j)/(m - 1)$ for the star in position $j \in \{1, \dots, m\}$ of that order; for $m = 1$, set $a(\varphi) = 0$. At each expansion, select the unexpanded star maximizing

$$A(S, g, \varphi) = a(\varphi) + \eta_n b_\varphi, \qquad \eta_n = \eta_0 \frac{\kappa}{\kappa + n},$$

where $\eta_0 \ge 0$ sets the initial guide strength, $\kappa > 0$ sets its decay scale, and n counts completed action evaluations in the current execution. Ties follow the baseline order. This makes the guide strongest before execution evidence is available and reduces its influence as observations accumulate. Setting $\eta_0 = 0$ recovers the baseline star order. Every selected star is still expanded exactly once, and the fixpoint loop is unchanged.

For an executor that instead keeps ready actions from multiple stars together, the same prior can be used directly through $f_{\text{guide}}(S, g, \mu) = f(S, g, \mu) + \eta_n b_\varphi$ for $\mu = (\varphi, h)$. This is an alternative integration, not an additional effect claimed for the single-star ready set of Algorithm 6. In particular, the guide does not replace the learned cost-to-go or reinterpret a semantic preference as a predicted execution cost.

Example. For $g = \text{HighRisk}(v)$, the transfer and shared-infrastructure rules may both survive the dependency filter and have the same symbolic relevance. If the supplied domain descriptions indicate that the shared infrastructure is a personal device, the LLM may initially favor that route as suggestive of related customer behavior. If it instead denotes a public facility, the LLM may give it

a weaker preference. These are domain assumptions about which route to try first, not evidence that any customer is high-risk. The graph matches and every rule precondition are still checked exactly. If the descriptions or statistics do not support such a distinction, the appropriate output is zero. The intended benefit is a useful initial ordering when execution experience cannot yet distinguish the alternatives; improvement is not guaranteed.

Cost and evaluation. Each goal uses at most one guide request, completed before the main loop. Let c_{guide} include prompt preparation, the LLM request, and response validation. For guided runs, the reported total cost is $c_{\text{guide}} + \text{cost}(C)$, and end-to-end latency includes guide generation even if the goal is reached without an action evaluation. Monetary API charges, if applicable, are reported separately from runtime. To isolate the guide's contribution, compare the same policy and execution engine with $\eta_0 = 0$ and $\eta_0 > 0$, holding all other settings fixed. Report guide-generation time, subsequent reasoning time, total latency, and action counts on new-rule or new-domain workloads, with η_0 and κ specified. The five main methods of Section 6 do not require this optional guide.

Correctness and limitations. The guide changes ordering only. All retained stars are eventually expanded unless the goal has already been derived, no action is discarded because of its preference, and suspended actions retain the same fixpoint treatment. Exact validation preserves soundness, and complete candidate discovery with fair execution preserves goal-level completeness and termination. These properties do not imply an efficiency guarantee: a misleading guide can increase reasoning cost, and even a useful ordering may fail to offset the initial LLM-call overhead.